

NetVis: A Composable Architecture for Large-Scale Network Visualization

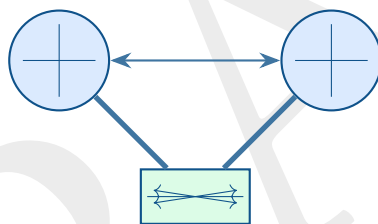
Technical Reference

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.5.0

Last updated: March 11, 2026



Abstract. NetVis is a Rust-based network topology layout and visualization engine comprising 90,000+ lines of code. It transforms complex multi-layer network topologies into publication-quality visualizations using composable layout algorithms, R^* -tree-accelerated label placement, force-directed edge bundling, and adaptive level-of-detail rendering. This document is the definitive technical reference for NetVis. It describes the data model, architecture, eight layout engines, ten edge routing and post-layout refiners, parallel edge rendering (five modes with automatic LAG/bond aggregation), interface rendering, node shape library, edge gradients, path emphasis, metric encoding, rendering pipeline, topology diffing, temporal analysis, traffic animation, annotation markup, interactive editing, visual effects, graph analytics, filter DSL, 16 quality metric families, and public API. Appendices provide the full YAML topology schema, performance baselines with comparative benchmarks against Graphviz, and a complete configuration reference.

Version	Date	Changes
1.5.0	March 2026	Subway layout; TreeLayout; interface rendering; node shape library; path emphasis; metric encoding; edge gradients; additional refiners (Starburst, Deoverlap, EdgeClipping, SmartLabel, LabelNudging, BoundaryBundling); terminal renderer
1.4.0	March 2026	Parallel edges (5 modes); Bond group/LAG aggregation; Orthogonal layout; Clarity scoring; Graph analytics; Filter DSL; Fuzzy validation
1.3.0	March 2026	Margin Insights; Cover Diagrams; Design Rule Primitives
1.2.0	March 2026	Integrated W2P Architectural Strategy; Formalized Type Logic
1.1.0	March 2026	Routing refiners, topology diffing, timeline, traffic animation, annotations, editor, visual effects, expanded quality metrics (16 families), and configuration reference appendix
1.0.0	Jan 2025	Initial Formal Pipeline Specification

Todo list

DRAFT

Contents

List of Design Decisions & Rules	2
1 Introduction	3
1.1 Design Philosophy	3
1.2 Audience	4
1.3 How to Read This Document	4
2 NetVis by Example	4
2.1 Example 1: Basic Campus Edge	4
2.2 Example 2: Geographic CDN	5
2.3 Example 3: Isometric Multi-Layer	6
2.4 Example 4: Parallel Edges and Bond Groups	7
2.5 Example 5: Topology Diff	8
2.6 Example 6: Subway Layout	9
2.7 Example 7: Constraint-Based Layout	10
3 Data Model	11
3.1 Graph Types	11
3.2 Node Data	11
3.3 Edge Data	11
3.4 Edge Groups	12
3.5 Topology Input Format	12
4 Architecture	12
4.1 Design Decision: Scene Graph as Intermediate Representation	13
4.2 Design Decision: Trait-Based Composition	13
5 Layout Algorithms	14
5.1 Force-Directed Layout	14
5.2 Hierarchical Layout (Sugiyama)	14
5.3 Geographic Layout	15
5.4 Isometric Multi-Layer Layout	15
5.5 Orthogonal Layout	15
5.6 Radial Tree Layout	16
5.7 Subway Layout	16
5.8 Tree Layout	16
5.9 Composite Layout	17
6 Parallel Edge Rendering	17
6.1 Problem	17
6.2 Design	18
6.3 Automatic Bandwidth Aggregation	18
7 Node Shapes and Device Icons	18
7.1 Geometric Shapes	19
7.2 Network Device Icons	19
8 Interface Rendering	20
8.1 Interface Data Model	20
8.2 Interface Status	20
8.3 Visual Modes	20
8.4 Interface Markers	20
9 Edge Gradients and Path Emphasis	21
9.1 Edge Gradients	21
9.2 Path Emphasis	21

10	Metric Encoding	22
10.1	Color Palettes	22
10.2	Mapping Configuration	22
11	Label Placement	23
11.1	Problem	23
11.2	Design Options	23
11.3	Decision	23
11.4	Algorithm	24
11.5	Performance	24
12	Edge Bundling	24
12.1	Problem	24
12.2	Bundling Modes	24
12.3	FDEB Implementation	24
13	Edge Routing and Post-Layout Refiners	25
13.1	Edge Routing	25
13.2	Overlap and Spacing	25
13.3	Label Refiners	26
13.4	Group Layout	26
14	Topology Diffing	26
15	Temporal Analysis	26
15.1	Snapshot Discovery	26
15.2	Entity History	26
15.3	CLI Subcommands	26
16	Traffic Flow Animation	26
16.1	Data Model	26
16.2	Rendering	27
17	Annotation Markup	27
18	Interactive Editor	27
18.1	Command Pattern	27
18.2	Web Component	27
19	Visual Effects	27
19.1	Effect Registry	28
19.2	Built-In Effects	28
19.3	Theme Adaptation	28
20	Graph Analytics	29
20.1	Metrics	29
20.2	Usage	29
21	Filter DSL	29
21.1	Architecture	29
21.2	Usage	29
22	Input Validation	30
22.1	Schema Validation	30
22.2	Fuzzy Field Correction	30
23	Quality Metrics	30
23.1	Metrics	30
23.2	Clarity Score	31
23.3	Topology Linting	31

23.4	CI Regression Detection	31
24	Rendering Pipeline	31
24.1	SVG Renderer	31
24.2	Terminal Renderer.	31
24.3	Level of Detail	31
24.4	Viewport Culling.	32
24.5	Interactive HTML Export	32
24.6	DOT Export.	32
24.7	Progressive Rendering	32
24.8	Theme Gallery.	32
25	Pluggable Node Renderer	33
25.1	Architecture.	33
26	API and Usage	33
26.1	CLI Usage	33
26.2	WASM / JavaScript	34
26.3	Web Component	34
26.4	API Quick Reference	35
27	Deployment	35
27.1	Deterministic Mode.	35
28	Performance	35
29	Limitations	36
29.1	Scale Ceiling at ~10,000 Nodes	36
29.2	Greedy Label Placement	37
29.3	Single-Threaded WASM	37
29.4	Non-Determinism Without Explicit Mode.	37
29.5	Adapter Coverage	37
29.6	No User Study Validation	37
30	Future Work	37
A	Topology YAML Schema	38
B	Configuration Reference	39
B.1	CLI Flags	39
B.2	CLI Subcommands	42
B.3	JSON Configuration File.	42
VLAN	Virtual Local Area Network	26
LAG	Link Aggregation Group	7

DRAFT

1 Introduction

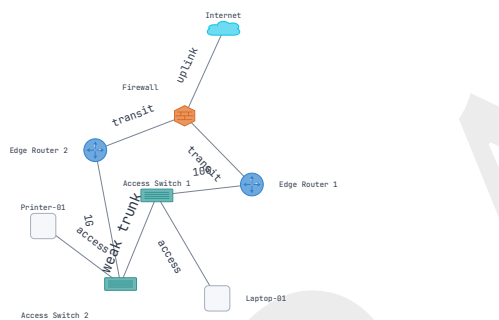
NetVis is a network topology layout and visualization engine. Given a graph of network devices and connections (from a YAML file, NetBox [1] instance, or LLDP discovery), it produces publication-quality SVG diagrams, interactive HTML visualizations, or PNG/PDF exports. It addresses the gap between general-purpose graph layout tools (Graphviz [2], OGDF [3], D3.js [4]) and manual diagramming applications (yEd [5], Visio).

1.1 Design Philosophy

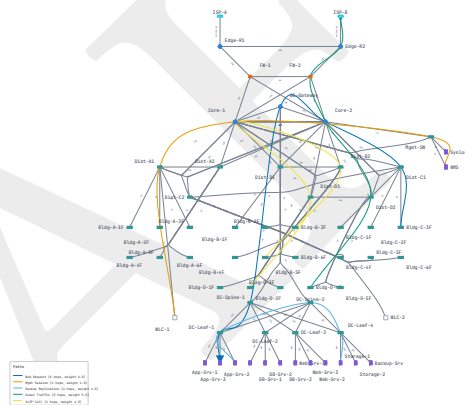
Two principles guide every design decision in NetVis:

Domain-specialized composition. Network topologies have exploitable structural regularity—hierarchical tiers, geographic site placement, protocol-layer stacking. Rather than using a single general-purpose layout algorithm, NetVis decomposes visualization into a pipeline of composable refiners, each specialized for one aspect of the output.

Quality by default. The system should produce readable, labeled, styled diagrams without manual post-processing. Quality metrics (crossings, overlaps, label readability) are measured as part of the rendering pipeline, not as an afterthought.

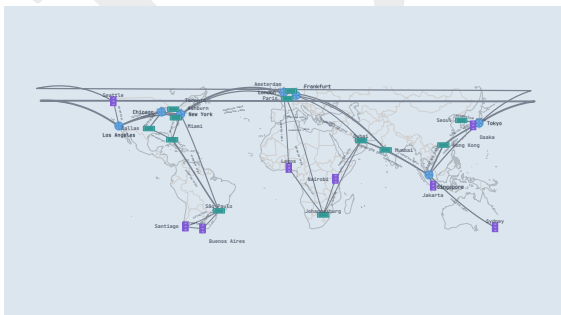


(a) Campus edge (force-directed, 8 nodes)

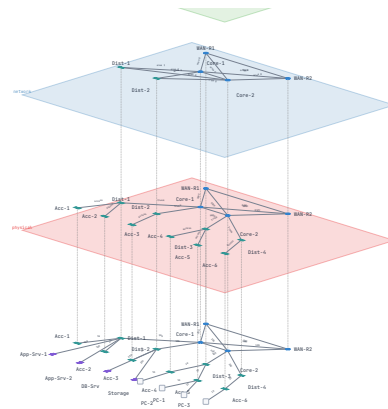


(b) Enterprise campus (72 nodes, edge bundling)

Figure 1. Representative NetVis outputs (1/3). All diagrams generated automatically from YAML with no manual adjustment.

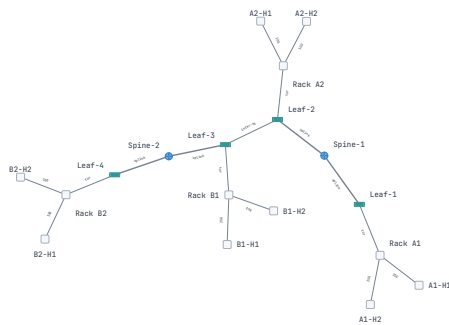


(a) Global CDN (geographic, great-circle arcs, map layer)

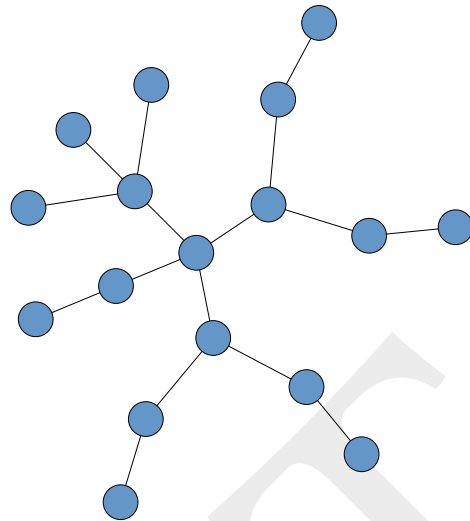


(b) Protocol stack (isometric multi-layer, 3 planes)

Figure 2. Representative NetVis outputs (2/3).



(a) Hierarchical datacenter (Sugiyama-style layered layout)



(b) Hub-and-spoke WAN (radial tree layout)

Figure 3. Representative NetVis outputs (3/3).

1.2 Audience

This document is intended for:

- Engineers embedding NetVis as a library in a network management platform (via Rust, Python, or WASM).
- Contributors to the NetVis codebase.
- Researchers who want to understand the algorithms in detail.

Familiarity with basic graph theory is assumed. Familiarity with network engineering concepts (spine-leaf, OSPF, BGP) is helpful but not required.

1.3 How to Read This Document

- **Quick start:** Read §2 (By Example), then §26 (API and Usage).
- **Data model and architecture:** §3–§4.
- **Deep dive into algorithms:** §5–§23.
- **Analysis features:** §14–§18 (diff, timeline, traffic, annotations, editor).
- **Deployment and configuration:** §27 and Appendix B.
- **Known issues:** §29.

2 NetVis by Example

This section presents seven topologies of increasing complexity, each illustrating different NetVis capabilities. All examples can be rendered with a single CLI command.

2.1 Example 1: Basic Campus Edge

A minimal 8-node campus network demonstrating the core YAML schema (Appendix A): node types, edge attributes, directed links, and force-directed layout (§5).

Basic Campus Edge

```
name: Basic campus edge
render:
  layout: force_directed
  theme: network
  seed: 123
nodes:
  - id: inet
```

```

label: Internet
node_type: cloud
- id: fw1
  label: Firewall
  node_type: firewall
- id: r1
  label: Edge Router 1
  node_type: router
  attributes: { role: edge, site: hq }
- id: s1
  label: Access Switch 1
  node_type: switch
- id: h1
  label: Laptop-01
  node_type: host
edges:
- from: inet
  to: fw1
  label: uplink
  directed: true
  attributes: { bandwidth_gbps: 10 }
- from: fw1
  to: r1
  label: transit
- from: r1
  to: s1
  label: 10G
- from: s1
  to: h1
  label: access

```

```
netvis -input basic.yaml -o basic.svg
```

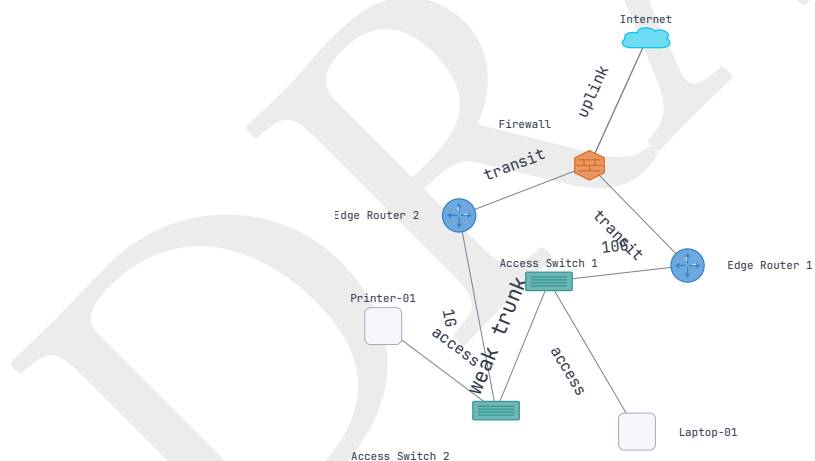


Figure 4. Rendered output of Example 1: an 8-node campus edge topology with force-directed layout, automatic label placement, and device-type icons.

2.2 Example 2: Geographic CDN

A 30-node global CDN using geographic layout (§5) with real-world coordinates, background map layer, and great-circle arc edges.

Global CDN Network (excerpt)

```

name: Global CDN Network
render:

```

```

preset: geographic
theme: dark
width: 1800
height: 1000
geographic:
  show_map: true
  great_circle_arcs: true
nodes:
  - id: nyc
    label: New York
    node_type: router
    lat: 40.7128
    lon: -74.0060
    highlight: true
    attributes: { region: north-america, tier: primary }
  - id: london
    label: London
    node_type: router
    lat: 51.5074
    lon: -0.1278
    highlight: true
  - id: tokyo
    label: Tokyo
    node_type: router
    lat: 35.6762
    lon: 139.6503
edges:
  - from: nyc
    to: london
    label: submarine cable
    edge_type: submarine
    attributes: { bandwidth_gbps: 400 }

```

```
netvis -input cdn.yaml -o cdn.svg
```

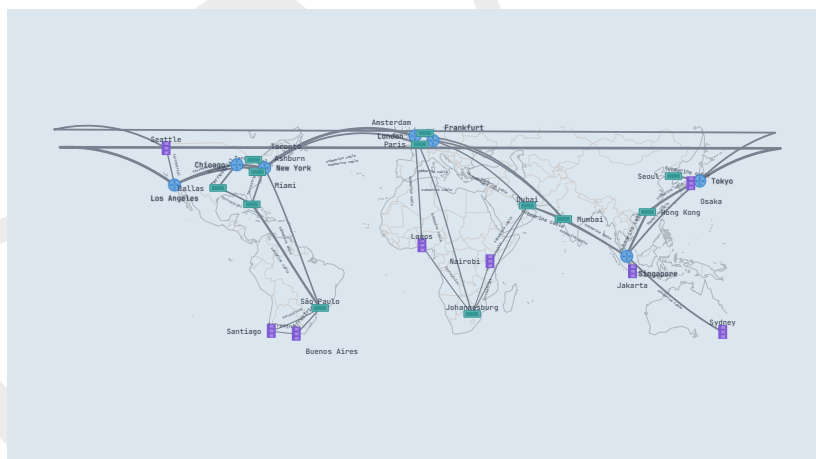


Figure 5. Rendered output of Example 2: a global CDN with geographic layout, great-circle arc edges, and a background map layer.

2.3 Example 3: Isometric Multi-Layer

A 44-node enterprise WAN visualized as three isometric protocol layers (§5): Physical (all devices), Network (L2/L3 only), and Routing (OSPF routers only).

Isometric Multi-Layer (excerpt)

```

name: Isometric multi-layer enterprise WAN
render:
  layout: isometric

```

```

width: 1800
height: 1400
theme: network
layers:
- id: physical
  label: Physical
  order: 0
- id: network
  label: Network
  order: 1
- id: routing
  label: Routing
  order: 2
nodes:
- id: p-core1
  label: Core-1
  node_type: router
  attributes: { layer: physical }
- id: n-core1
  label: Core-1
  node_type: router
  attributes: { layer: network }
edges:
- { from: p-core1, to: p-dist1, label: "10G" }
- { from: n-core1, to: n-dist1, label: "trunk" }

```

```
netvis -input multilayer.yaml -o multilayer.svg
```

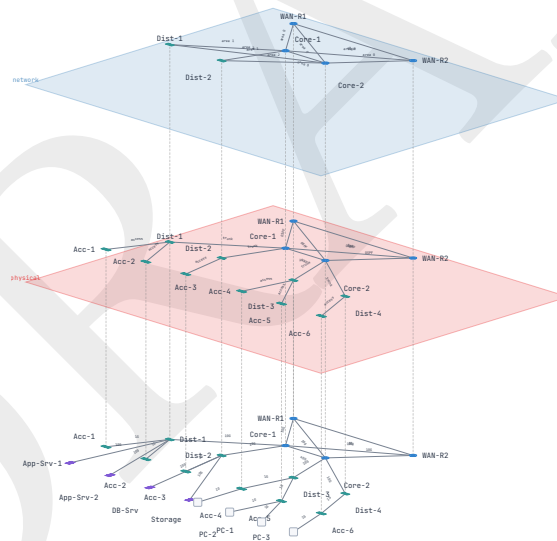


Figure 6. Rendered output of Example 3: isometric multi-layer visualization with three protocol planes (Physical, Network, Routing) separated along the Z-axis.

2.4 Example 4: Parallel Edges and Bond Groups

A datacenter spine-leaf topology demonstrating all five parallel edge rendering modes (§6) and automatic Link Aggregation Group (LAG) bandwidth aggregation.

Parallel Edges with Bond Groups (excerpt)

```

name: Parallel Edges Showcase
render:
  preset: hierarchical
  theme: network

```

```

nodes:
- id: core1
  label: Core-R1
  node_type: router
- id: core2
  label: Core-R2
  node_type: router
- id: dist1
  label: Dist-SW1
  node_type: l3_switch
edges:
  # LAG: 4x 10G links, auto-aggregated to 40G
- from: core1
  to: core2
  label: "10G"
  bond_group: core-lag
  parallel_mode: bundled
- from: core1
  to: core2
  label: "10G"
  bond_group: core-lag
  # Redundant uplinks shown as offset lines
- from: core1
  to: dist1
  label: "10G"
  parallel_mode: offset
- from: core1
  to: dist1
  label: "10G"

```

```
netvis -input parallel.yaml -o parallel.svg
```

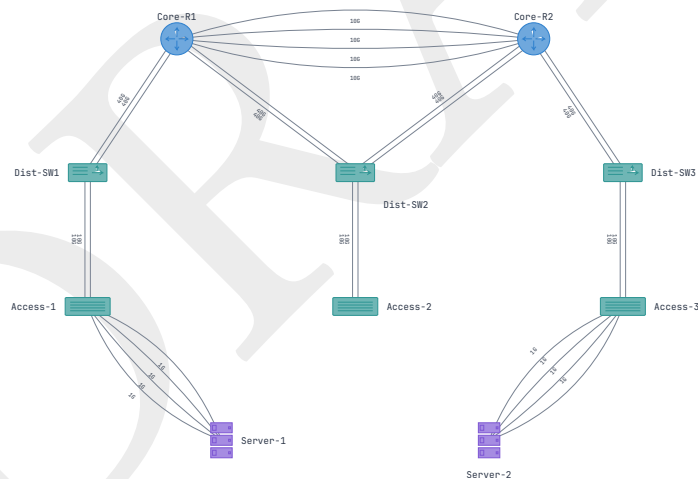


Figure 7. Rendered output of Example 4: four tiers of a datacenter topology demonstrating Bundled (core LAG), Offset (redundant uplinks), BondingIcon (bonded access), and Stacked (server NICs) parallel edge modes.

2.5 Example 5: Topology Diff

NetVis can structurally compare two topologies and produce a visual diff showing added, removed, and modified elements.

Topology Diff

```
netvis diff old.yaml new.yaml -o diff.svg
```

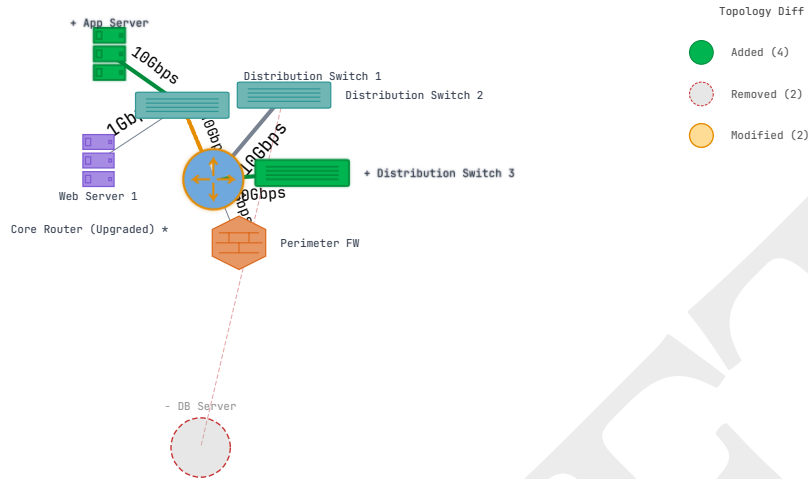


Figure 8. Topology diff output. Added elements are shown in green, removed elements in red/dimmed, and modified elements in amber.

2.6 Example 6: Subway Layout

A transit-map–style visualization (§5.7) constraining all edges to 0, 45, or 90 degrees and snapping nodes to a uniform grid. Ideal for metro-style network documentation.

Subway Layout

```
name: Subway topology showcase
render:
  layout: subway
  theme: blueprint
  width: 1600
  height: 1000
nodes:
  - id: hub-central
    label: Central Hub
    node_type: router
  - id: ring-north
    label: North Ring
    node_type: switch
  - id: ring-south
    label: South Ring
    node_type: switch
  - id: edge-east
    label: East Edge
    node_type: router
edges:
  - from: hub-central
    to: ring-north
    label: 100G
  - from: hub-central
    to: ring-south
    label: 100G
  - from: ring-north
    to: edge-east
    label: 10G
```

```
netvis -input subway-showcase.yaml -o subway.svg
```

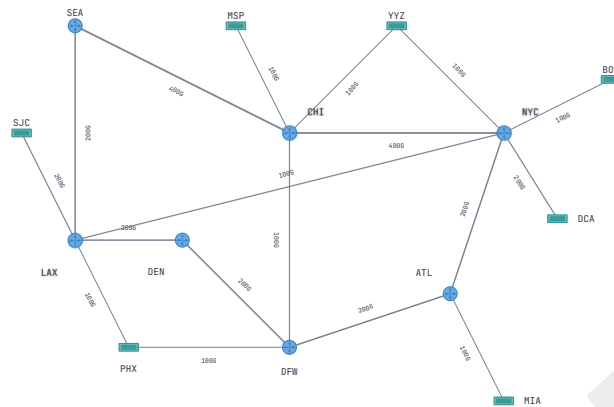


Figure 9. Rendered output of Example 6: subway-style layout with octilinear edge routing (0/45/90 degree constraints) and grid-snapped node placement. The blueprint theme complements the geometric precision of the layout.

2.7 Example 7: Constraint-Based Layout

NetVis supports declarative placement constraints that override or augment the layout engine's automatic positioning. Constraints can pin nodes to absolute coordinates, enforce alignment, or set minimum separation distances.

Constraint-Based Layout (excerpt)

```
name: Constraint showcase
render:
  layout: force_directed
  theme: network
nodes:
  - id: core1
    label: Core-1
    node_type: router
    constraints:
      pin: { x: 600, y: 200 } # Absolute position
  - id: core2
    label: Core-2
    node_type: router
    constraints:
      align_with: core1 # Same Y as core1
      min_separation: 200 # At least 200 units from core1
  - id: dist1
    label: Dist-1
    node_type: l3_switch
    constraints:
      below: core1 # Place below core1
edges:
  - from: core1
    to: core2
  - from: core1
    to: dist1
```

```
netvis -input constraints.yaml -o constraints.svg
```

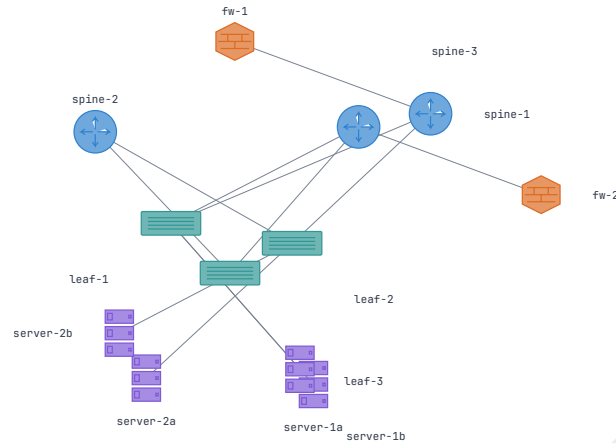


Figure 10. Rendered output of Example 7: constraint-based layout with pinned positions, alignment rules, and minimum separation constraints. Force-directed simulation respects declared constraints while freely positioning unconstrained nodes.

3 Data Model

NetVis uses a directed graph as its core data structure, built on the petgraph library [6].

3.1 Graph Types

The core types are:

- `NetVisGraph` — a `StableGraph<NodeData, EdgeData>` from petgraph. `StableGraph` is used (rather than `Graph`) because node and edge indices remain valid after removal—important for the editor’s undo/redo system.
- `NodeId` — a `NodeIndex<u32>` stable across graph mutations.
- `EdgeId` — an `EdgeIndex<u32>` stable across graph mutations.

3.2 Node Data

Each node carries:

- `id`: `String` — unique identifier (e.g., "router-fra-01")
- `label`: `Option<String>` — display label
- `node_type`: `Option<String>` — device type (drives icon and default styling)
- `attributes`: `HashMap<String, Value>` — arbitrary metadata
- `position`: `Option<Point2D>` — optional layout hint
- `group`: `Option<GroupId>` — group membership
- `layer`: `Option<LayerId>` — layer membership (isometric)

Note

The `node_type` field drives device-specific rendering: icon shape, default color, and default size. The full set of built-in node types is described in §7.

3.3 Edge Data

Each edge carries:

- `label`: `Option<String>` — edge label
- `weight`: `f64` — default 1.0
- `edge_type`: `Option<String>` — semantic type (p2p, trunk, submarine, etc.)
- `directed`: `bool` — default false
- `bond_group`: `Option<String>` — LAG/bond group identifier

- `attributes: HashMap<String, Value>` — arbitrary metadata

3.4 Edge Groups

When multiple edges connect the same pair of nodes, NetVis groups them into an `EdgeGroup`. Key fields:

- `edge_ids: Vec<EdgeId>` — member edges
- `count: usize` — number of parallel edges
- `total_bandwidth: Option<String>` — parsed aggregate (e.g., "40G")
- `is_bonded: bool` — true when a `bond_group` is set
- `parallel_mode: ParallelMode` — rendering mode (see §6)

The `total_bandwidth` field is computed automatically by parsing per-link bandwidth labels (e.g., four edges labeled "10G" produce "40G").

3.5 Topology Input Format

The primary input format is YAML:

Listing 1. Example topology (campus network)

```
name: Campus Edge Network

render:
  preset: default
  theme: network
  width: 1200
  height: 800

nodes:
  - id: internet
    label: Internet
    node_type: cloud

  - id: fw
    label: Perimeter FW
    node_type: firewall

  - id: core-sw
    label: Core Switch
    node_type: l3_switch

  - id: srv1
    label: Web Server
    node_type: server

edges:
  - from: internet
    to: fw
    label: 1Gbps

  - from: fw
    to: core-sw
    label: 10Gbps

  - from: core-sw
    to: srv1
    label: 1Gbps
```

4 Architecture

NetVis is organized as a six-layer pipeline:

1. **Adapters.** Stateless converters that ingest topology data from external sources (YAML, Net-Box [1] REST API, LLDP/CDP JSON) and produce a `NetVisGraph`. Each adapter implements the `Adapter` trait with three methods: `fetch`, `validate`, `to_graph`.

2. **Layout engines.** Pluggable algorithms behind a common Layout trait. A layout takes a NetVisGraph and returns a Scene—a flat list of positioned nodes, edges (as Bézier paths), and label anchors. Eight engines ship: force-directed (Standard and Enterprise), hierarchical (Sugiyama [7]), radial tree, geographic (Mercator), isometric multi-layer, orthogonal, subway (octilinear), tree, and composite (multi-region).
3. **Refiners.** Post-layout transformations on the scene graph. Each refiner implements LayoutRefiner and may add, remove, or reposition scene elements. Current refiners include: label placement (R^* -tree-accelerated), edge bundling (FDEB), constraint solving, overlap removal, edge nudging, label nudging, edge clipping, smart label snapping, starburst grouping, boundary bundling, port docking, and orthogonal edge routing (see §13).
4. **Renderers.** An SVG renderer (primary), a Canvas2D renderer (for interactive browser use), and an ASCII terminal renderer (netvis show). All support level-of-detail suppression and viewport culling.
5. **Export.** SVG output is optionally converted to PNG (via resvg), PDF (via svg2pdf), DOT (for Graphviz interoperability), or self-contained interactive HTML (with embedded WASM binary and pan/zoom controls).
6. **Analysis.** Post-render modules: topology diffing (§14), temporal snapshots (§15), traffic flow animation (§16), SVG annotation markup (§17), and graph analytics (§20).

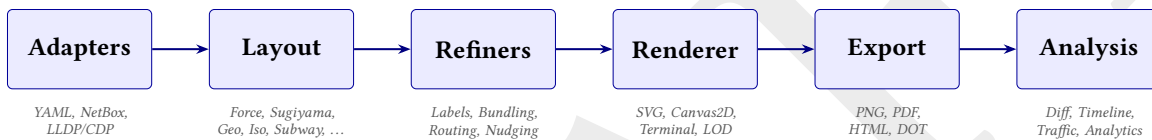


Figure 11. The NetVis six-layer pipeline.

4.1 Design Decision: Scene Graph as Intermediate Representation

Problem: Layout algorithms produce coordinates, but renderers need styled, labeled geometry. Where does styling happen?

Options considered:

- (a) Style during layout (couples styling to layout code).
- (b) Style during rendering (couples styling to renderer).
- (c) Use an intermediate representation (Scene) that carries positioned, styled geometry.

Decision: Option (c). The Scene struct carries fully positioned nodes (with shapes, colors, labels) and edges (as Bézier paths). Refiners transform the scene between layout and rendering.

Consequences: Positive: renderers are simple (they just draw what the scene contains). Negative: the scene must carry all rendering data, increasing memory usage. At 5,000 nodes, the scene consumes approximately 9.7 MB of heap.

4.2 Design Decision: Trait-Based Composition

Problem: How should layout algorithms and refiners be composed?

Decision: All layout engines implement trait Layout, all refiners implement trait LayoutRefiner. A LayoutPipeline chains a layout engine with zero or more refiners. In YAML, this is expressed as:

Listing 2. Pipeline composition in YAML

```

render:
  layout: force_directed
  refiners:
    - labels
    - bundling
    - nudging
    - constraints
  
```

Consequences: Positive: new algorithms and refiners can be added without modifying existing code. Negative: the trait boundary prevents cross-component optimization (e.g., bundling cannot influence label placement during the same pass). We mitigate this with `reposition_edge_labels()`, which re-places labels after bundling.

5 Layout Algorithms

NetVis ships eight layout engines, each implementing the `Layout` trait.

5.1 Force-Directed Layout

5.1.1 Problem

Position nodes in 2D such that connected nodes are close and unconnected nodes are separated. The general case has no closed-form solution; iterative simulation is standard.

5.1.2 Design Options

- (a) **Standard** ($O(n^2)$): Compute all-pairs repulsion directly [8]. Simple, deterministic, good for small graphs.
- (b) **Barnes-Hut** ($O(n \log n)$): Approximate far-field repulsion using a spatial tree [9]. Faster for large graphs but adds tree-building overhead.

5.1.3 Decision

NetVis ships both (a) and (b) and automatically selects between them. Graphs under 2,000 nodes use the Standard engine (where quadtree overhead exceeds repulsion savings); larger graphs use the Enterprise engine.

5.1.4 Implementation

The Standard engine wraps the `fjadra` Rust library (Verlet integration [10] with spring-charge forces). The Enterprise engine is a custom implementation using a Barnes-Hut quadtree [9] with opening angle $\theta = 0.9$. On non-WASM targets, force calculation is parallelized via Rayon.

Key parameters:

- Velocity damping: $\gamma = 0.6$
- Alpha decay: $1 - 0.001^{1/300}$
- Iteration count: $\min(300, 50 + \lfloor n/10 \rfloor)$
- Minimum edge length: 60 pt (enforced post-simulation)

5.1.5 Usage

Listing 3. Force-directed layout in YAML

```
render:
  layout: force_directed
  seed: 42
```

5.2 Hierarchical Layout (Sugiyama)

5.2.1 Problem

Visualize DAG-structured or tiered networks (e.g., spine-leaf datacenters) with nodes arranged in horizontal layers and minimized edge crossings.

5.2.2 Implementation

The layout follows the four-phase Sugiyama framework [7]:

1. **Cycle removal.** Reverse edges to make the graph acyclic.
2. **Layer assignment.** Longest-path-first heuristic.
3. **Crossing minimization.** Barycenter or median heuristic (configurable).
4. **Coordinate assignment.** Assign x-coordinates to minimize edge bends.

Performance: 19 ms at 1,000 nodes.

Listing 4. Hierarchical layout in YAML

```
render:
```

```
preset: hierarchical
```

5.3 Geographic Layout

5.3.1 Problem

Visualize WAN or multi-site topologies with nodes placed at their real-world geographic coordinates.

5.3.2 Implementation

Nodes with `lat/lon` attributes are projected using an equirectangular projection with dynamic scale normalization: $s = 1000 / \max(\Delta x, \Delta y)$. Edges are rendered as geodesic great-circle arcs (Vincenty approximation, 20 segments).

Antimeridian Wrapping

Edges crossing $\pm 180^\circ$ longitude exhibit a projection discontinuity. NetVis detects jumps exceeding 150 projected pixels and renders two copies of the path: one in the primary projection and one shifted by 360° . This is a pragmatic heuristic, not a formal solution.

5.4 Isometric Multi-Layer Layout

5.4.1 Problem

Visualize protocol stacks or layered architectures with a pseudo-3D perspective.

5.4.2 Implementation

Each layer runs an independent force-directed sub-layout. Results are projected isometrically with rotation angle $\phi = \pi/10$ (18°) and configurable layer spacing (default: 350 scene units). Layer planes render as filled SVG polygons.

Note

SVG layer planes must use RGB hex color + separate `fill-opacity` attribute (not 8-digit hex), because some SVG renderers double-apply alpha from 8-digit hex.

5.5 Orthogonal Layout

5.5.1 Problem

Network diagrams in engineering documentation traditionally use axis-aligned (Manhattan-style) edges. Force-directed and Sugiyama layouts produce diagonal edges that feel informal in standards-compliant documentation.

5.5.2 Implementation

The orthogonal layout engine assigns node positions to a grid and routes all edges as sequences of horizontal and vertical segments with configurable corner radius. A sweep-line algorithm minimizes total edge length while maintaining minimum channel spacing between parallel routes.

5.5.3 Usage

Listing 5. Orthogonal layout in YAML

```
render:
  layout: orthogonal
```

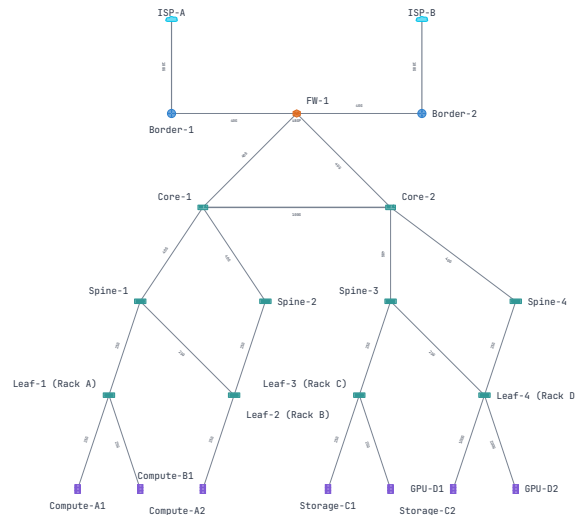


Figure 12. Orthogonal layout: all edges route as horizontal and vertical segments with rounded corners. Suitable for standards-compliant engineering documentation.

5.6 Radial Tree Layout

Best for hub-and-spoke topologies. Places a root node at the center with children radiating outward in concentric rings. Ring radii scale with subtree size to prevent overlap.

Listing 6. Radial layout in YAML

```
render:
  preset: radial
```

5.7 Subway Layout

5.7.1 Problem

Transit-map-style diagrams constrain edges to a small set of directions (0, 45, and 90 degrees) and snap nodes to a regular grid. This style clarifies topological relationships in ring and meshed network backbones.

5.7.2 Implementation

The subway layout engine assigns nodes to grid positions using a force-directed initialization followed by grid snapping. Edges are then routed along the octilinear grid: each edge is bent at most twice, and the routing algorithm minimizes total bend count. Node positions are iteratively adjusted to minimize edge crossings while respecting the grid constraint.

The **blueprint theme** is recommended: its technical ink style complements the geometric precision of octilinear routing. The example file `subway-showcase.yaml` demonstrates a full ring backbone.

5.7.3 Usage

Listing 7. Subway layout in YAML

```
render:
  layout: subway
  theme: blueprint
```

5.8 Tree Layout

A basic tree layout for strict hierarchies where the input graph is guaranteed to be a tree (or forest). Places the root at the top and arranges children in balanced subtree columns. Faster than Sugiyama for pure trees because it skips cycle removal and crossing minimization.

Listing 8. Tree layout in YAML

```
render:
  layout: tree
  tree_root: core1      # Optional: specify root node ID
```

5.9 Composite Layout

5.9.1 Problem

Real-world networks combine different structural paradigms: a WAN backbone has geographic placement, while each site's internal fabric is hierarchical. A single layout algorithm cannot serve both.

5.9.2 Implementation

Assigns different layout algorithms to different sub-graphs within a single canvas. A `CompositeConfig` maps group IDs to layout engines; a parent-level layout positions groups relative to each other.

5.9.3 Usage

Listing 9. Composite multi-region layout in YAML

```
render:
  layout: composite
  composite:
    parent_layout: force_directed
    groups:
      - id: wan
        layout: geographic
      - id: dc-east
        layout: hierarchical
      - id: dc-west
        layout: hierarchical
```

When used with the `--cluster` CLI flag, NetVis applies community detection (Louvain, label propagation, or edge betweenness) to automatically partition the graph into groups, which are then visualized with group boundary overlays.

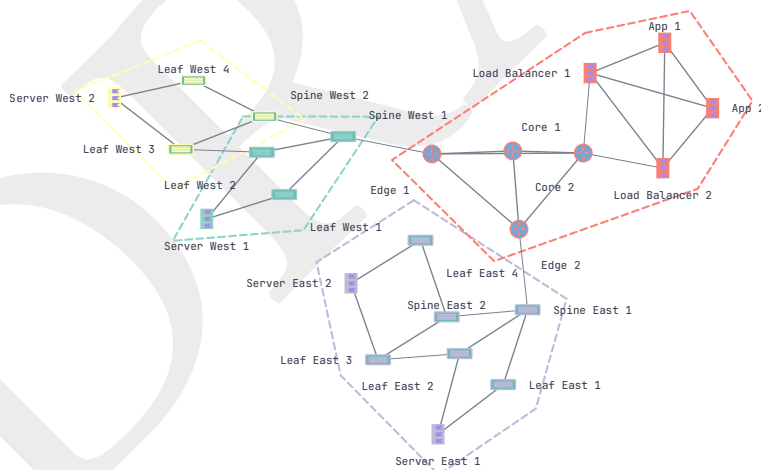


Figure 13. Automatic clustering with community-detection group boundaries. The Louvain algorithm partitions nodes into communities; each community receives a distinct color overlay and boundary polygon.

6 Parallel Edge Rendering

6.1 Problem

Network topologies frequently contain multiple physical links between the same pair of devices—redundant uplinks, LAG bundles, and NIC-teaming configurations. General-purpose graph tools either render these as overlapping lines (unreadable) or collapse them into a single edge (losing information).

6.2 Design

NetVis provides five rendering modes for parallel edges, selectable per edge group via the `parallel_mode` YAML field or the `ParallelMode` enum:

Offset (default). Each link drawn as a visually separate parallel line with perpendicular offset.

Example: 4 edges produce offsets of -6 , -2 , $+2$, $+6$ pixels. Best for showing redundant paths.

Bundled. All links collapsed into a single thick line (width scales with $\sqrt{\text{count}}$) with an aggregate multiplicity label (e.g., “4x 10G (40G)”). Best for LAG groups where aggregate bandwidth matters more than individual links.

BondingIcon. Single line with a bracket or chain-link icon at the midpoint indicating bonding. Compact representation for bonded links.

Stacked. Lines drawn very close together (1–2 px offset), creating a “thick line” effect without merging strokes. Best for NIC-teaming visualizations.

Angled. Curves with different angles/curvatures. Distinguishes many parallel links without excessive width.

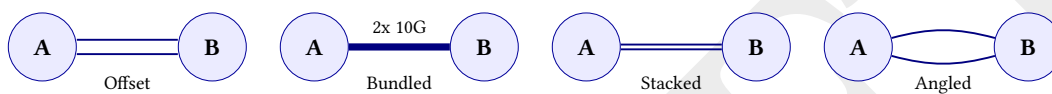


Figure 14. The five parallel edge rendering modes (BondingIcon omitted for clarity). Each mode trades visual fidelity for compactness.

6.3 Automatic Bandwidth Aggregation

When edges share a `bond_group` identifier, the system automatically parses per-link bandwidth labels and computes aggregate totals:

- **Bundled mode:** Shows total only (e.g., “40G”).
- **Other bonded modes:** Shows detail (e.g., “4x 10G (40G)”).
- **LAG-prefixed:** When a `bond_group` is set, the label is prefixed with the group name: “LAG core-lag 40G”.

The `parse_bandwidth_gbps()` helper recognizes common suffixes (G, Gbps, M, Mbps) and the `format_bandwidth_gbps()` function selects the most readable unit for the aggregate.

Listing 10. Bond group configuration

```
edges:
- from: core1
  to: core2
  label: "10G"
  bond_group: core-lag
  parallel_mode: bundled
- from: core1
  to: core2
  label: "10G"
  bond_group: core-lag
- from: core1
  to: core2
  label: "10G"
  bond_group: core-lag
- from: core1
  to: core2
  label: "10G"
  bond_group: core-lag
# Rendered as single thick line: "LAG core-lag 40G"
```

7 Node Shapes and Device Icons

NetVis supports a rich library of geometric shapes and network device icons for node rendering.

7.1 Geometric Shapes

The following geometric shapes are available via the shape field in the node style or via node_type:

Circle. Default shape for generic nodes.

Rectangle. Axis-aligned rectangle; useful for hosts and servers.

Ellipse. Horizontal ellipse for cloud or WAN nodes.

RoundedRectangle. Rectangle with configurable corner radius.

Diamond. Rotated square; used for decision points or firewalls.

Triangle. Upward-pointing triangle.

Hexagon. Six-sided polygon; useful for AS or BGP speakers.

7.2 Network Device Icons

The following network-specific icon shapes are registered by default:

Router. Cylindrical icon with routing arrows.

Switch. Rectangular icon with port stubs.

L3Switch. Switch icon with routing overlay.

Firewall. Shield or brick-wall icon.

Server. Rack-unit rectangle.

Cloud. Cumulus cloud outline.

LoadBalancer. Funnel or balance icon.

Database. Stacked-cylinder icon.

Endpoint. Laptop/workstation icon.

WirelessAp. Access-point radio wave icon.

Unknown node_type values render as circles. Custom node renderers can be registered programmatically (see §25).

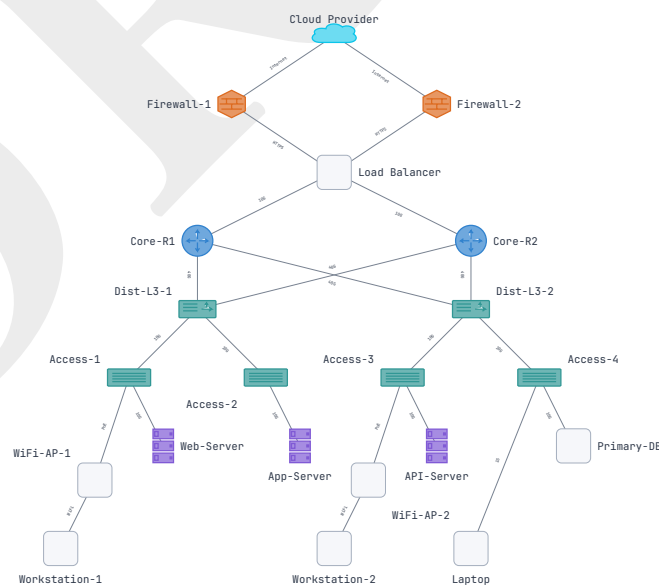


Figure 15. Full device icon library. Each icon shape adapts its size, fill, and stroke to the active theme.

8 Interface Rendering

NetVis treats network interfaces as first-class visual objects, not merely as edge endpoints. Each node can expose its interfaces with configurable visual modes and status indicators.

8.1 Interface Data Model

Interfaces carry:

- `id`: String — interface identifier (e.g., "eth0")
- `label`: Option<String> — display label
- `status`: InterfaceStatus — operational state
- `mode`: InterfaceMode — visual rendering mode
- `marker`: InterfaceMarker — endpoint marker shape

8.2 Interface Status

Up. Interface is operationally up; rendered in the active color.

Down. Interface is operationally down; rendered in muted gray.

AdminDown. Interface is administratively disabled; rendered with a strikethrough or suppressed icon.

Errors. Interface is up but reporting errors; rendered in amber.

8.3 Visual Modes

Dot. A small filled circle on the node boundary. Compact; suitable for large topologies where per-port detail would clutter the diagram.

Port. A rectangular port stub protruding from the node boundary, labeled with the interface name. Suitable for switch and server nodes where physical port layout matters.

Detailed. Full interface card with status color, label, speed, and operational state. Suitable for close-up datacenter rack diagrams.

8.4 Interface Markers

None. No endpoint marker on the edge at this interface.

Dot. Small circle at the edge endpoint.

Square. Small square marker; indicates a dedicated interface.

Diamond. Diamond marker; indicates a peering or WAN interface.

Listing 11. Interface configuration

```
nodes:
- id: core-sw
  label: Core Switch
  node_type: switch
  interfaces:
  - id: eth0
    label: "eth0"
    status: up
    mode: port
    marker: dot
  - id: eth1
    label: "eth1"
    status: errors
    mode: port
    marker: square
  - id: mgmt0
    label: "mgmt0"
    status: admin_down
    mode: dot
    marker: none
```

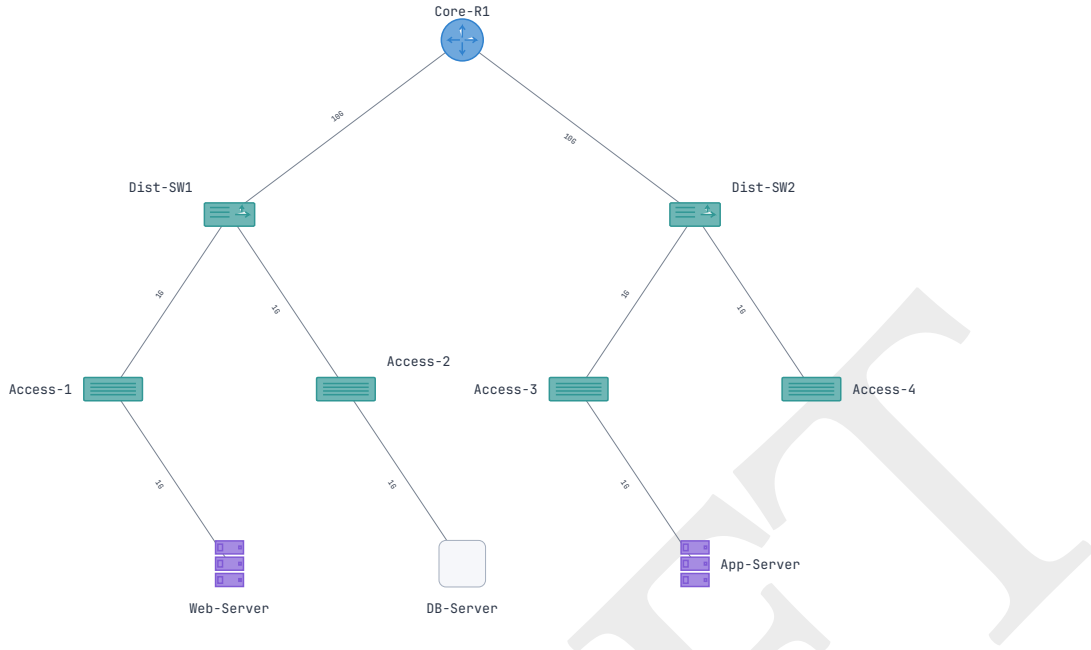


Figure 16. Interface rendering showcase. The three visual modes (Dot, Port, Detailed) and four status colors (Up, Down, AdminDown, Errors) are demonstrated across representative network device types.

9 Edge Gradients and Path Emphasis

9.1 Edge Gradients

Directional stroke gradients encode flow direction or metric values along an edge without requiring separate annotation. A gradient edge transitions smoothly from a *start color* at the source endpoint to an *end color* at the target endpoint.

Listing 12. Edge gradient configuration

```
edges:
- from: core1
  to: edge1
  label: uplink
  gradient:
    start_color: "#3b82f6" # blue at source
    end_color: "#ef4444" # red at target
```

Gradients are implemented as SVG `<linearGradient>` definitions with the gradient vector aligned to the edge direction, ensuring correct orientation regardless of node positions.

9.2 Path Emphasis

Path emphasis highlights a subset of edges as a named network path, distinguishing it from background topology with increased weight, color, and optionally an animated flow indicator.

Two emphasis levels are provided:

Primary. Full-weight highlight; the path is clearly dominant. Used for the active forwarding path or critical circuit.

Secondary. Reduced-weight highlight; the path is visible but subordinate. Used for backup paths or alternative routes.

Three fanout modes control how an emphasized path interacts with parallel edges:

Fixed. All parallel edges in a group receive the same emphasis level.

Scaled. Emphasis weight scales with the number of parallel edges, producing a thicker highlight for larger LAG bundles.

Hybrid. Primary members receive full emphasis; secondary members receive reduced emphasis.

Listing 13. Path emphasis configuration

```
paths:
- id: primary-path
  label: "Primary forwarding path"
  emphasis: primary
  fanout_mode: scaled
  nodes: [ core1, dist1, access1, host1 ]

- id: backup-path
  label: "Backup path"
  emphasis: secondary
  fanout_mode: fixed
  nodes: [ core2, dist2, access1, host1 ]
```

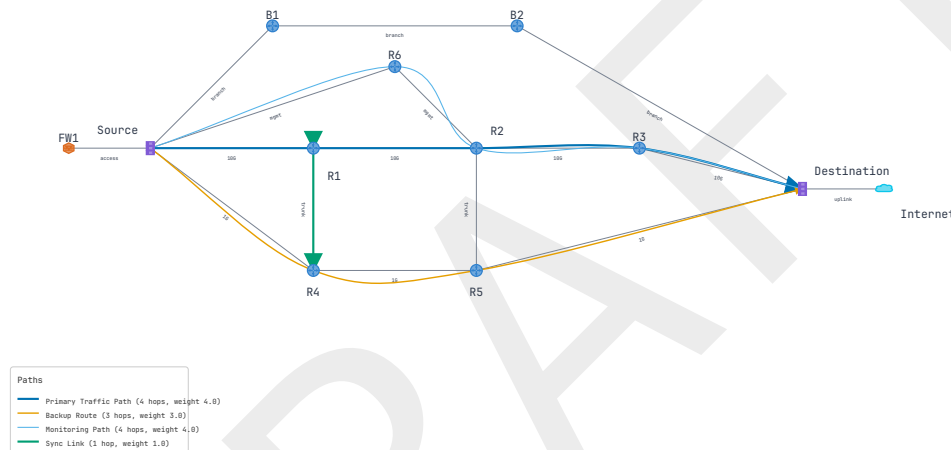


Figure 17. Path emphasis showcase. Primary paths are rendered with full-weight highlight color; secondary paths with reduced emphasis. Fanout mode *Scaled* is shown on a LAG bundle.

10 Metric Encoding

NetVis supports data-driven visualization by mapping arbitrary node attributes to visual channels (color, size, opacity). This transforms static topology diagrams into live network dashboards.

10.1 Color Palettes

The following sequential palettes are built in for metric-to-color mapping:

- blues — light blue to deep blue
- greens — light green to deep green
- oranges — pale orange to burnt orange
- reds — light red to crimson
- purples — lavender to deep purple
- greys — light grey to near-black

All palettes are perceptually uniform and work on both light and dark themes.

10.2 Mapping Configuration

Listing 14. Metric encoding in YAML

```
render:
  metric_encoding:
```

```

- attribute: cpu_utilization
  channel: fill_color
  palette: reds
  min: 0.0
  max: 1.0
- attribute: packet_loss
  channel: size
  min_size: 15
  max_size: 40
- attribute: link_utilization
  channel: edge_color
  palette: oranges

```

Listing 15. CLI metric encoding

```
netvis --input topology.yaml --metric cpu_utilization --palette reds
```

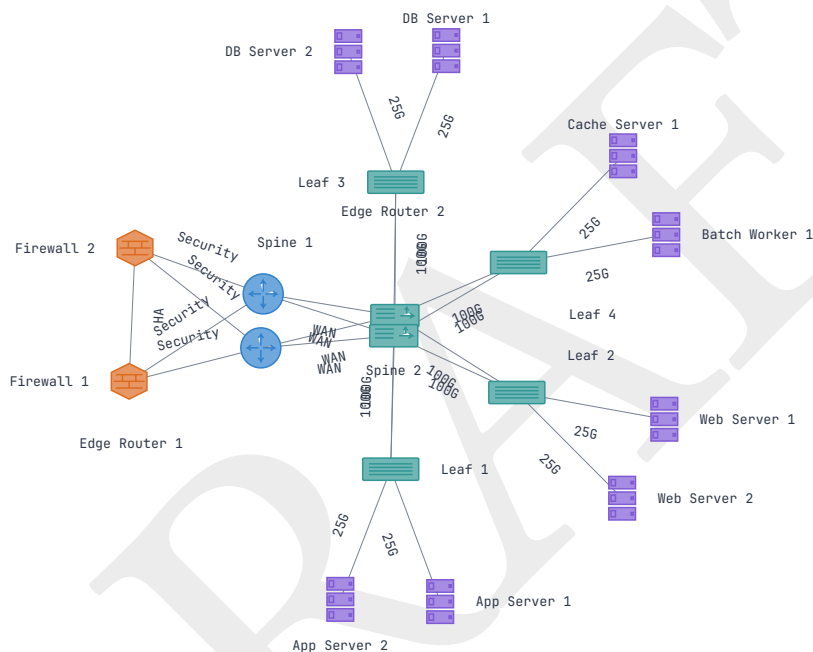


Figure 18. Metric encoding showcase. Node fill color encodes CPU utilization (reds palette); node size encodes interface error count; edge stroke color encodes link utilization (oranges palette).

11 Label Placement

11.1 Problem

Place text labels near their owner (node or edge) without overlapping other labels, nodes, or edges. This is a variant of the NP-hard point-feature label placement (PFLP) problem [11].

11.2 Design Options

- No collision avoidance** (Graphviz [2] approach): place labels at fixed positions; accept overlaps.
- $O(n^2)$ **greedy**: For each label, check all previously placed labels for collision.
- R*-tree-accelerated greedy**: Use a spatial index for $O(\log n)$ collision queries.

11.3 Decision

Option (c). The R*-tree [12] build cost (~ 5 ms) is recouped after approximately 20 collision queries, which occurs by ~ 50 labels. The spatial index is implemented via the rstar crate [13].

11.4 Algorithm

NetVis uses a two-phase greedy placement:

Phase 0 (edge labels): Edge labels are placed first because they are more constrained—each must sit near its edge’s midpoint. For curved edges, the midpoint is computed via arc-length sampling. A deviation threshold $\delta = 0.08$ prevents labels from clustering at bundle convergence points.

Phase 1 (node labels): Node labels are placed around each node, avoiding already-placed edge labels. For each node, the algorithm evaluates 8 candidate positions (4 cardinal + 4 diagonal) and selects the highest-scoring one.

Scoring function:

$$S(c, \ell) = \exp(-\alpha \cdot d(c, \ell)) - \beta \sum_{r \in R(c)} A_{\text{overlap}}(c, r) \cdot w(r)$$

Where:

- $d(c, \ell)$: distance from candidate position to label owner
- $\alpha = 1.0$: distance decay rate
- $\beta = 300.0$: overlap penalty weight
- $w(r) = 5.0$ for labels/nodes (hard collision), 0.3 for edge strokes (soft collision)

Edge exclusion zone optimization: Curved/bundled edges can produce 47,000+ segment AABBs. NetVis skips per-segment exclusion zone registration for non-straight edges, registering only a single bounding-box exclusion per curved path.

11.5 Performance

Nodes	Time	Overlaps	Avg. Distance
100	<1 ms	0	8.2 pt
500	12 ms	0	9.1 pt
1000	48 ms	2	10.3 pt
5000	210 ms	18	12.7 pt

Table 1. Label placement performance on spine-leaf topologies.

12 Edge Bundling

12.1 Problem

Dense graphs produce visual clutter from crossing edges. Edge bundling groups spatially compatible edges into bundles, reducing clutter.

12.2 Bundling Modes

NetVis supports two bundling modes via the `BundlingMode` enum:

GeometricProximity (default). Force-directed edge bundling (FDEB): edges are pulled toward each other based on geometric proximity and a compatibility score. Best for large, dense graphs.

SharedEndpoint. Groups edges that share a common source or target node into a visible bundle. Simpler and faster than FDEB; best for hub-and-spoke topologies.

12.3 FDEB Implementation

NetVis implements a variant of Holten and Van Wijk’s Force-Directed Edge Bundling (FDEB) [14]. Our variant differs from the original in three ways:

1. **Singularity guard:** Force model uses $f = k_c / (d + 1.0)$ (not $d + 0.1$), preventing hairball artifacts when compatible edges nearly overlap.

2. **Spring preservation:** Spring coefficient $1.5k$ (not $0.5k$) maintains edge shape under bundling.
3. **Post-processing:** Catmull-Rom spline (tension 0.4) + 3–5 passes of Laplacian smoothing ($\alpha = 0.5$) + decimation filter (drop points <15 px apart).

In addition to the standard geometric bundling, the FDEB subsystem includes a **circuit board (Manhattan/PCB) mode** that routes bundled edges along horizontal and vertical channels, producing a printed-circuit-board aesthetic suitable for detailed rack and patch-panel diagrams.

Note

Bundling runs *after* label placement. A `reposition_edge_labels()` pass re-places labels on edges whose curvature changed significantly. This ordering is intentional: labels should avoid bundled paths, not the other way around.

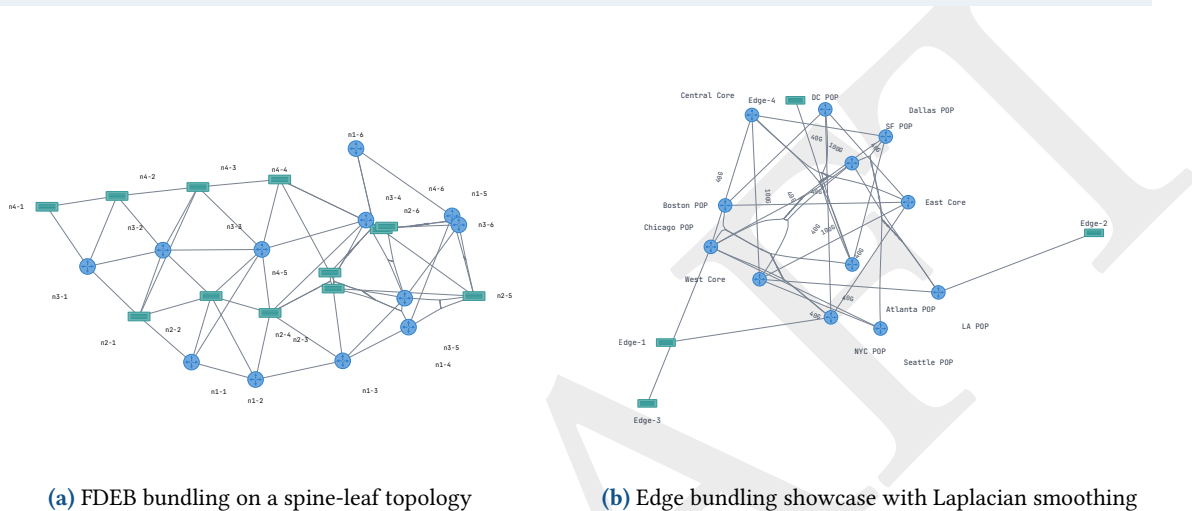


Figure 19. Force-directed edge bundling (FDEB) reduces visual clutter by grouping spatially compatible edges. Post-processing with Catmull-Rom splines and Laplacian smoothing produces clean, readable bundles.

13 Edge Routing and Post-Layout Refiners

NetVis provides ten post-layout refiners, each implementing the `LayoutRefiner` trait. Refiners are composable and can be chained in any order.

13.1 Edge Routing

SmartRoutingRefiner. Bends edges around node and label obstacles using an R^* -tree-based visibility graph with A^* pathfinding. Resulting waypoints are smoothed into Bézier splines.

OrthogonalRoutingRefiner. Converts edges to axis-aligned Manhattan paths with configurable corner radius. Used automatically by the orthogonal layout and available as a standalone refiner.

PortDockingRefiner. Adjusts edge endpoints to dock at specific interface/port positions on node boundaries, distributing connections evenly around the perimeter.

BoundaryBundlingRefiner. Bundles edges at composition group boundaries, routing them through a shared crossing point at the group boundary and fanning out inside.

13.2 Overlap and Spacing

DeoverlapRefiner. Detects overlapping nodes and pushes them apart using a repulsive spring force. Configurable margin (default: 10 scene units) and maximum iteration count. Runs after initial layout to resolve any remaining overlaps without disturbing the global structure.

EdgeClippingRefiner. Clips edge endpoints to the exact boundary of each node's shape. Without clipping, edges terminate at node centers; this refiner ensures arrowheads and connectors sit correctly on the node perimeter regardless of shape (circle, rectangle, diamond, etc.).

NudgingRefiner. Detects overlapping or nearly-overlapping edges and nudges them apart for visual clarity. Particularly useful after parallel edge grouping where offset modes need fine-tuning.

13.3 Label Refiners

SmartLabelRefiner. 8-position snapping around nodes: evaluates all four cardinal and four diagonal positions and selects the position with the highest clearance score. Applied after initial greedy placement for a second-pass improvement.

LabelNudgingRefiner. Iterative repulsive force between overlapping label bounding boxes. Each iteration moves overlapping labels apart by a small fraction of their overlap area. Converges in 10–30 iterations for typical topologies.

13.4 Group Layout

StarburstRefiner. Positions nodes radially within containment groups (AS, Virtual Local Area Network (VLAN), or site groups). Each group's nodes are arranged in a circle around the group centroid, scaled to the group size. Best for AS-level topology diagrams where group structure matters more than individual node distances.

14 Topology Diffing

NetVis can structurally compare two NetVisGraph instances and produce a visual diff. The diff module computes:

- Added, removed, and modified nodes (by ID and attribute comparison).
- Added, removed, and modified edges.

The `apply_diff_styles()` function color-codes the rendered scene: green for added elements, red/dimmed for removed, amber for modified. A diff legend is automatically inserted.

Listing 16. CLI topology diff

```
netvis diff old.yaml new.yaml -o diff.svg
```

15 Temporal Analysis

The timeline module tracks topology evolution over time.

15.1 Snapshot Discovery

`discover_snapshots()` scans a directory of YAML topology files, extracts timestamps from filenames or metadata, and returns them in chronological order as `SnapshotEntry` values.

15.2 Entity History

`build_node_history()` and `build_edge_history()` trace an entity's lifecycle across snapshots: when it first appeared, all attribute changes, and whether it has been removed. Results are returned as `EntityHistory` with a vector of `ChangeRecord` entries.

15.3 CLI Subcommands

Listing 17. Timeline CLI

```
# List all snapshots in a directory
netvis timeline --dir snapshots/

# Query history of a specific node
netvis query --dir snapshots/ --node router-fra-01
```

16 Traffic Flow Animation

The traffic module injects animated flow overlays into rendered SVGs.

16.1 Data Model

Each edge can carry traffic metadata:

Listing 18. Traffic configuration

```
traffic:
  edges:
    - id: edge-core-to-agg
      utilization: 0.75
      direction: bidirectional
      style: dot
```

16.2 Rendering

`animate_svg()` post-processes the SVG string to inject:

- CSS @keyframes for dot/dash flow along edge paths.
- Utilization-to-color mapping: green (low) → amber → red (congested).
- Play/pause and speed controls as an SVG overlay.
- Zero-traffic edges receive CSS dimming (`opacity: 0.3`).

17 Annotation Markup

The annotation module injects text annotations and callouts into rendered SVGs.

Listing 19. Annotation configuration

```
annotations:
  - anchor: router-fra-01
    type: callout
    text: "Primary BGP peer"
    shape: box
    layer: notes
```

Annotations support layer filtering (`show_only_layers`, `hidden_layers`), pointer lines from callout to anchor node, and shapes: circle, box, and arrow.

18 Interactive Editor

The editor module provides interactive topology editing for browser deployments via the WASM target.

18.1 Command Pattern

All mutations are represented as Command objects: `MoveNodeCommand`, `TransactionCommand`, etc. Each command stores only a delta (e.g., 24 bytes for a node move—source and destination coordinates), not a full graph snapshot. An `UndoStack` with 50-action capacity provides undo/redo.

Note

Delta-based undo uses ~41,000x less memory than snapshot-based undo for a 5,000-node topology.

18.2 Web Component

A Lit-based `<netvis-editor>` Web Component wraps the WASM editor:

Listing 20. Web Component usage

```
<netvis-editor
  topology="campus.yaml"
  layout-algorithm="force-directed"
  read-only="false">
</netvis-editor>
```

Provides declarative attributes, an imperative API (`setTopology()`, `resetView()`, `exportSVG()`), and JSX type augmentations for React/Vue integration.

19 Visual Effects

NetVis provides a pluggable visual effects system that maps semantic effect names to SVG filter definitions.

19.1 Effect Registry

The EffectRegistry deduplicates identical filter definitions via content-addressed hashing. Effects are assigned to nodes or edges via the highlight field in YAML:

Listing 21. Assigning effects in YAML

```
nodes:
- id: core-router
  highlight: true           # resolves to "glow-selection"
- id: alert-node
  highlight: "emphasis"    # resolves to "glow-emphasis"
- id: combo-node
  highlight: "stack:drop-shadow-subtle+glow-selection"
```

19.2 Built-In Effects

Name	Type	Description
drop-shadow-subtle	Drop shadow	Light offset shadow
drop-shadow-emphasis	Drop shadow	Strong offset shadow
inner-shadow-boundary	Inner shadow	Inset shadow for group boundaries
glow-selection	Glow	Selection highlight
glow-status	Glow	Status indicator
glow-emphasis	Glow	Emphasis marker
stack:<a>+	Composed	Combine any two base effects

Table 2. Built-in visual effects.

19.3 Theme Adaptation

All effects adapt to the active theme. Glow intensity, rim sharpness, and shadow direction vary by theme family:

- **Light/Print:** Crisp rim, slightly stronger glow.
- **Dark/Network:** Softer, reduced intensity (no rim).
- **Blueprint:** Technical ink-like rim with soft halo.
- **High-contrast:** Tighter blur, strongest opacity.

Effect Budget

SVG filter performance degrades at ~50+ filtered elements. The `effects.max_filtered_elements` config key (default: 50) sets a budget; exceeding it emits a BudgetExceeded warning and auto-disables further filters.

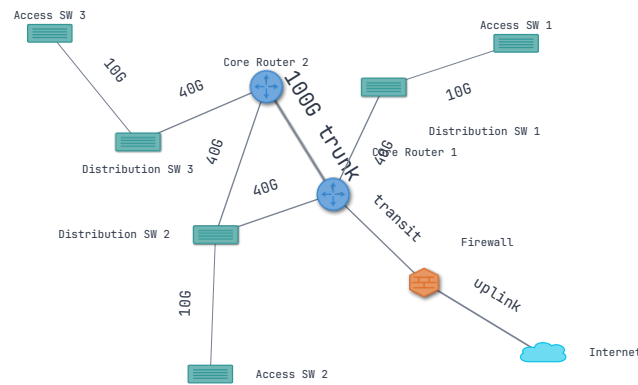


Figure 20. Visual effects showcase (light theme). Nodes demonstrate drop shadows, glow selection, emphasis glow, and composed `stack:drop-shadow+glow` effects. All effects adapt their intensity and color to the active theme.

20 Graph Analytics

The analytics module computes structural graph metrics and injects them into the rendered output as a visual “Network Insights” card.

20.1 Metrics

Betweenness centrality. Identifies nodes that serve as critical transit points. Computed via Brandes’ algorithm ($O(nm)$).

Graph diameter. The longest shortest path between any two nodes. Highlights the “worst-case” latency path in the topology.

Clustering coefficient. Measures how densely interconnected each node’s neighbors are. High coefficients indicate redundant paths.

20.2 Usage

Listing 22. Analytics overlay

```
netvis --input topology.yaml --analytics -o analytics.svg
```

Results are injected into the Scene metadata and optionally rendered as a summary card in the SVG output.

21 Filter DSL

NetVis includes a domain-specific language for filtering topology elements before rendering.

21.1 Architecture

The filter system comprises three layers:

1. **Parser.** Converts filter expressions into an AST. Supports boolean operators (and, or, not), attribute selectors (`type == "router"`), glob patterns (`id = "core-*`), and set operations (`group in ["dc-east", "dc-west"]`).
2. **Evaluator.** Walks the AST against each node/edge, returning a boolean include/exclude decision.
3. **LLM layer (optional).** Feature-gated natural language interface that translates English queries (“show me only the core routers”) into filter AST nodes.

21.2 Usage

Listing 23. Filter DSL examples

```
# Show only routers
netvis --input topology.yaml --filter 'type == "router"'

# Show core devices in dc-east
netvis --input topology.yaml \
  --filter 'group == "dc-east" and id ~= "core-*"'
```

22 Input Validation

22.1 Schema Validation

Topology and configuration files are validated against a JSON Schema (generated from Rust types via schemars). Schema files can be exported for IDE autocomplete:

Listing 24. Schema export

```
netvis schema --type topology > topology-schema.json
```

22.2 Fuzzy Field Correction

The `validation::fuzzy` module provides “Did you mean...?” suggestions for misspelled YAML field names using string similarity (Levenshtein distance via the `strsim` crate). For example, typing `nod_type` instead of `node_type` produces:

```
Warning: Unknown field 'nod_type' at nodes[0].
Did you mean: node_type?
```

Structured diagnostics are returned as `ValidationDiagnostic` entries with severity, location, and suggested corrections.

23 Quality Metrics

NetVis computes quality metrics as part of every render, following the aesthetic metrics framework of Purchase [15] and Dunne et al. [16], producing a `DiagnosticsEnvelope` alongside the output.

23.1 Metrics

NetVis computes 16 metric families, aggregated into a unified `AestheticScore` (0–100) via `src/quality/scoring.rs`:

Edge crossings. R*-tree-accelerated segment intersection testing ($O(E \log E)$). Crossings are weighted by angle: orthogonal crossings receive weight 1.0; acute crossings receive higher penalties.

Overlap area. AABB intersection between all label/node pairs. Sub-pixel overlaps (<2 px) are ignored.

Label readability. Composite of label-owner distance, font-size variance, and left-to-right reading order.

Angular resolution. Minimum angle between consecutive incident edges at each node.

Node alignment. Degree to which nodes align on axes or grid lines.

Edge clearance. Minimum distance from non-incident edges to nodes.

Coincident edges. Detection of duplicate or overlapping edge paths.

Contrast. Label color contrast against background (WCAG-derived).

Edge-length variance. Coefficient of variation across all edge lengths; lower is better.

Spatial distribution. Evenness of node distribution across the canvas (entropy-based).

Path straightness. Curvature penalty for bent edges.

Viewport coverage. Whether elements extend beyond viewport bounds.

Node-edge overlap. Non-incident edge segments crossing node bounding boxes.

Density heatmap. Spatial density heatmap for identifying visual hotspots.

23.2 Clarity Score

In addition to the fine-grained metrics above, NetVis computes a composite *Clarity Score* (0–100) based on four equally-weighted factors:

1. **Edge crossings:** Penalizes intersecting edges, weighted by crossing angle.
2. **Angular resolution:** Rewards even angular distribution of edges around each node.
3. **Node spacing:** Penalizes nodes that are too close together or too far apart relative to the median.
4. **Label overlap:** Penalizes overlapping label bounding boxes.

The Clarity Score is designed as a single-number summary for CI/CD pipelines and regression detection.

23.3 Topology Linting

Separate from aesthetic quality, the `lint` module performs structural topology checks: isolated nodes, disconnected subgraphs, duplicate edge IDs, and missing labels. Lint results appear as `Warning-severity` entries in the `DiagnosticsEnvelope`.

23.4 CI Regression Detection

Quality baselines are stored in `benches/regression_baselines.json`. CI fails if any metric degrades by more than 20%. This threshold accounts for runner variability while catching genuine regressions.

24 Rendering Pipeline

24.1 SVG Renderer

The primary renderer produces SVG using the `svg crate`. Elements are ordered by the painter’s model: background (map layers) → layer planes (isometric) → group boundaries → edges → nodes → labels → effects.

24.2 Terminal Renderer

The `netvis show` subcommand renders topology to ASCII art in the terminal. The terminal renderer uses Unicode box-drawing characters and ANSI color codes to produce readable diagrams without a graphical display. It respects the `--theme` flag (`light` produces plain ASCII; `dark` produces ANSI-colored output).

Listing 25. Terminal renderer

```
netvis show --input topology.yaml
netvis show --input topology.yaml --theme dark
```

24.3 Level of Detail

LOD suppresses visual details at configurable thresholds, following the approach of Abello et al. [17]:

Profile	Labels	Edge Lbl.	Interfaces	Effects
Conservative	2000	1000	1000	500
Balanced	500	300	300	200
Aggressive	200	100	100	50

Table 3. LOD suppression thresholds (elements suppressed when count exceeds value).

LOD is applied at render time without mutating the scene. “Auto” mode selects a profile based on node count.

24.4 Viewport Culling

For pan/zoom scenarios, elements outside the viewport (plus a configurable margin) are skipped during rendering. Applied at render time.

24.5 Interactive HTML Export

The `export::html` module generates self-contained HTML files with the WASM binary embedded as base64. The resulting file can be opened directly from `file://` without a web server, providing pan, zoom, and node-drag interactivity.

Listing 26. Interactive HTML export

```
netvis --input topology.yaml --format html -o topology.html
```

24.6 DOT Export

The `export::dot` module writes Graphviz DOT format for interoperability with the wider graph visualization ecosystem. This enables side-by-side comparison workflows and allows users to leverage Graphviz's extensive output format support.

Listing 27. DOT export

```
netvis --input topology.yaml --format dot -o topology.dot
```

24.7 Progressive Rendering

For very large topologies, the renderer can operate in progressive mode: elements are rendered in chunks with progress callbacks and cooperative cancellation. This prevents blocking the UI thread in WASM deployments.

24.8 Theme Gallery

NetVis ships with 7 built-in themes. Figure 21 shows the same enterprise campus topology rendered with four different themes, demonstrating how the rendering pipeline adapts colors, glows, shadows, and label styling to each theme family.

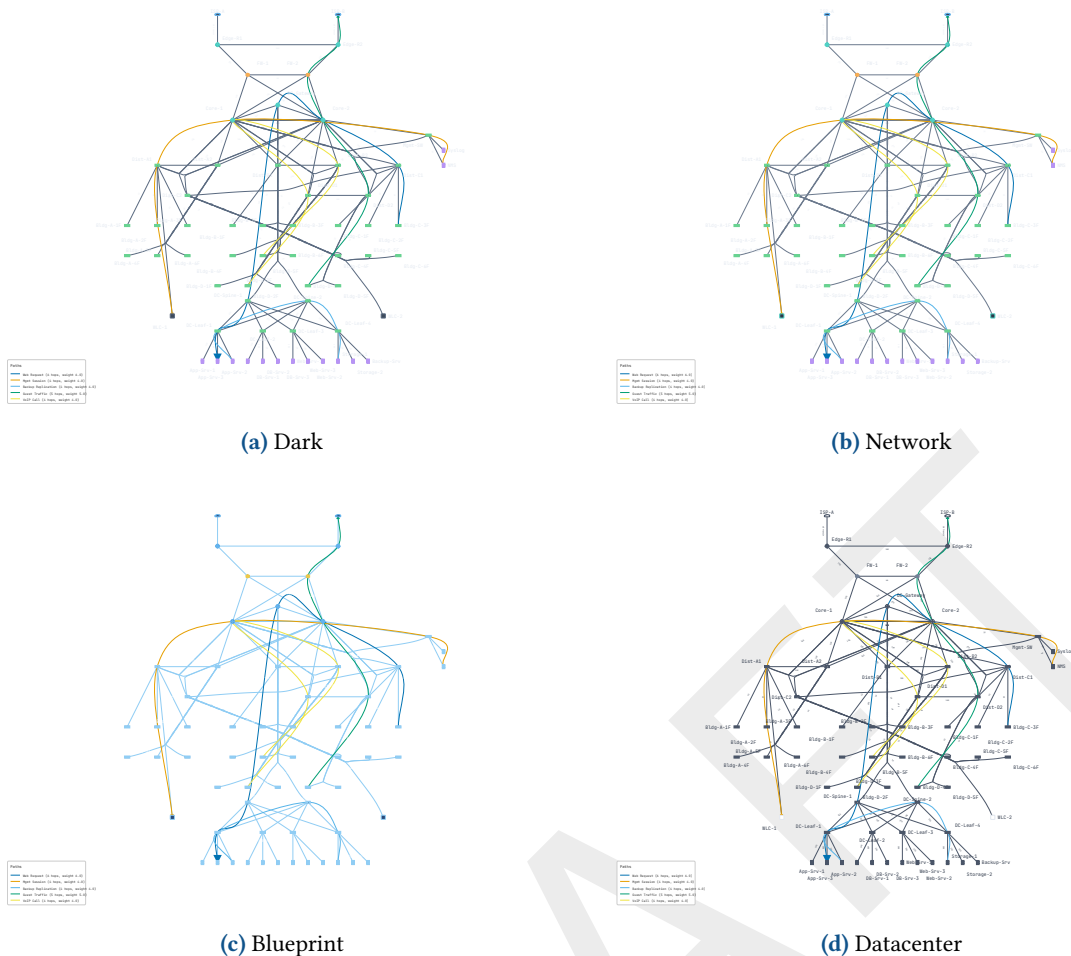


Figure 21. Theme comparison: the same topology rendered with four of NetVis’s 7 built-in themes. Each theme adapts node icons, edge colors, label styling, background, glow intensity, and shadow direction.

25 Pluggable Node Renderer

NetVis provides a trait-based plugin system for custom node rendering, allowing external crates to register custom node visualizations.

25.1 Architecture

The `NodeRenderer` trait defines the interface for custom node renderers. Built-in types (router, switch, server, firewall, cloud, `l3_switch`, and the full icon library described in §7) are registered at startup. Custom renderers can be added programmatically by implementing the trait and registering with the `NodeRendererRegistry`:

Listing 28. Custom node type registration (YAML side)

```
nodes:
- id: my-device
  node_type: my_custom_type # handled by registered custom renderer
  label: Custom Device
```

The `NodeRendererRegistry` maps `node_type` strings to renderer implementations. Unknown types fall back to the default circle renderer.

26 API and Usage

26.1 CLI Usage

Listing 29. Common CLI commands

```
# Render a topology
netvis --input topology.yaml -o output.svg
```

```

# Use hierarchical layout
netvis --input topology.yaml --preset sugiyama

# Dark theme, large canvas
netvis --input topology.yaml --theme dark --width 1920 --height 1080

# Subway layout with blueprint theme
netvis --input subway-showcase.yaml --layout subway --theme blueprint

# Export to PNG or interactive HTML
netvis --input topology.yaml --format png --dpi 150
netvis --input topology.yaml --format html -o topology.html

# Topology diff
netvis diff old.yaml new.yaml -o diff.svg

# Filter nodes before rendering
netvis --input topology.yaml --include-node "router-*" -o filtered.svg
netvis --input topology.yaml --path-from core --path-to edge -o path.svg

# Traffic animation overlay
netvis --input topology.yaml --traffic traffic.yaml -o animated.svg

# Annotation markup
netvis --input topology.yaml --annotations notes.yaml -o annotated.svg

# Metric encoding
netvis --input topology.yaml --metric cpu_utilization --palette reds

# Timeline and query subcommands
netvis timeline --dir snapshots/
netvis query --dir snapshots/ --node router-fra-01

# Terminal renderer
netvis show --input topology.yaml

```

26.2 WASM / JavaScript

Listing 30. JavaScript integration

```

import { init, renderToSVG, renderWithDiagnostics,
  createInteractive } from '@netvis/core';

await init();

// Static SVG
const svg = await renderToSVG(topologyJson);
document.getElementById('viz').innerHTML = svg;

// SVG + diagnostics (recommended for embedders)
const { output, diagnostics } = await renderWithDiagnostics(topologyJson);

// Interactive (pan, zoom, drag)
const viz = await createInteractive(topologyJson, '#viz-container');
viz.resetView();

```

A background Web Worker API (`layoutWorkerRun`) is also available for offloading layout computation with progress callbacks that stream intermediate node positions.

26.3 Web Component

A Lit-based `<netvis-editor>` Web Component provides a declarative integration path:

Listing 31. Declarative Web Component

```

<netvis-editor topology="campus.yaml"
  layout-algorithm="force-directed">
</netvis-editor>

```

See §18 for the imperative API.

26.4 API Quick Reference

Function	Input	Returns
<code>nv::render</code>	Graph, Options	RenderEnvelope
<code>Layout::layout</code>	Graph	Scene
<code>SvgRenderer::render</code>	Scene	SVG string
<code>render_to_png</code>	SVG, PngOptions	PNG bytes
<code>render_to_pdf</code>	SVG, PdfOptions	PDF bytes
<code>export_dot</code>	Scene	DOT string
<code>ExportBuilder::build</code>	Config	Multi-format output

Table 4. Core API entry points.

27 Deployment

NetVis compiles from a single Rust codebase to three targets:

Native CLI. `cargo build --release --features cli`. Full feature set including HTTP-based adapters (NetBox), parallel layout (Rayon), and file I/O.

WebAssembly [18]. `wasm-pack build --target web`. Single-threaded, no file I/O. Canvas + SVG rendering. Automatic engine selection and LOD apply as in native mode.

Python. `maturin build --features python`. PyO3 with ABI3 stability (Python 3.8+).

Feature gates (`cli`, `wasm`, `python`) control which dependencies are included. WASM-incompatible crates (`reqwest`, `tokio`, `rayon`) are gated to native-only.

27.1 Deterministic Mode

For CI/CD pipelines, deterministic mode ensures reproducible output by sorting all HashMap iterations, seeding the RNG, and fixing layout initialization:

Listing 32. Deterministic mode via CLI

```
netvis --input topology.yaml --deterministic -o output.svg
```

28 Performance

Measured on Apple M4 Pro, 24 GB RAM, macOS, Rust 1.93.

Operation	100	500	1K	2K	5K
Force-directed layout	20 ms	144 ms	309 ms	816 ms	1.75 s
Hierarchical layout	1.5 ms	8 ms	19 ms	—	—
SVG render	0.8 ms	4.5 ms	11 ms	76 ms	223 ms
Peak heap allocation	215 KB	952 KB	1.9 MB	3.8 MB	9.7 MB

Table 5. Performance baselines by topology size (spine-leaf topologies).

10,000-node spine-leaf (Enterprise engine, auto LOD): 5.3 s end-to-end.

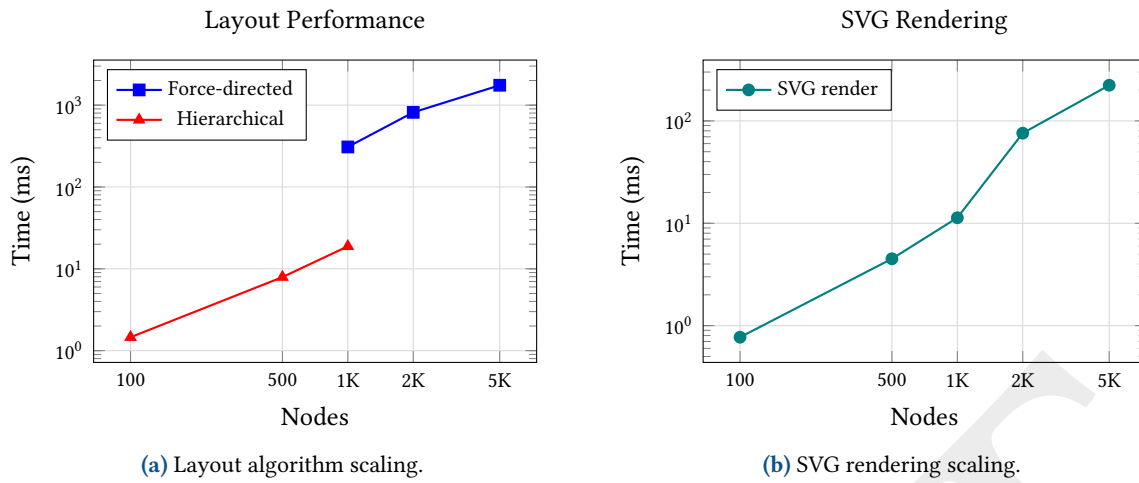


Figure 22. NetVis benchmark results on spine-leaf topologies. Left: layout computation time for force-directed (Barnes–Hut) and hierarchical (Sugiyama) engines. Right: SVG rendering time including label placement. Both axes are log-scaled; data points are median wall-clock times.

Tool	100 nodes	1K nodes	2K nodes
Graphviz sfdp	84 ms	250 ms	914 ms
Graphviz neato	88 ms	1,024 ms	4,434 ms
NetVis (full pipeline)	24 ms	330 ms	620 ms

Table 6. Comparative benchmark on identical spine-leaf topologies. NetVis times include layout, label placement, styling, and SVG rendering.

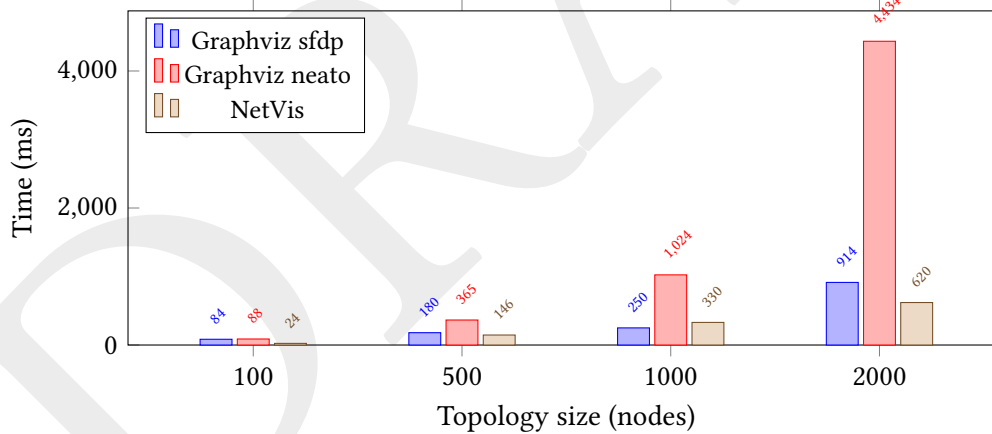


Figure 23. Comparative performance on identical spine-leaf topologies. NetVis includes the full pipeline (layout, labels, styling, SVG render); Graphviz includes layout and SVG output only. Apple M4 Pro, 24 GB RAM, best of 3 runs.

29 Limitations

29.1 Scale Ceiling at ~10,000 Nodes

Description: The Enterprise engine handles 10,000 nodes in ~5 s. Hierarchical and geographic layouts scale better, but 50,000+ nodes would benefit from GPU-accelerated force calculation.

Workaround: Use hierarchical, geographic, or tree layout (all scale better than force-directed), or apply LOD aggressively.

Planned resolution: GPU layout via WebGPU compute shaders is under investigation.

29.2 Greedy Label Placement

Description: Labels are placed sequentially; earlier placements constrain later ones. There is no global optimization. In pathological cases (very dense clusters), some labels may overlap.

Workaround: Suppress labels via LOD at high density, or increase canvas dimensions.

Planned resolution: A simulated-annealing post-pass is planned.

29.3 Single-Threaded WASM

Description: The WASM target cannot use Rayon for parallel force calculation. Large layouts are slower in the browser than natively.

Workaround: Use Web Workers for background layout computation (the WASM API supports this pattern).

Planned resolution: WebAssembly threads (SharedArrayBuffer) once browser support stabilizes.

29.4 Non-Determinism Without Explicit Mode

Description: By default, HashMap iteration order is non-deterministic. This affects layout reproducibility.

Workaround: Enable DeterministicMode via `-deterministic` flag or the `render.seed` YAML field.

29.5 Adapter Coverage

Description: Only YAML, NetBox, and LLDP/CDP adapters are built in. No SNMP, streaming telemetry, or intent-based sources.

Workaround: Export topology from your tool to YAML and feed it to NetVis.

Planned resolution: The Adapter trait is public; community adapters are encouraged.

29.6 No User Study Validation

Description: Quality claims are based on automated metrics (crossings, overlaps, readability scores), not validated by perceptual user studies with network engineers.

Planned resolution: A controlled user study comparing NetVis output with Graphviz and hand-drawn diagrams is planned.

30 Future Work

1. **GPU-accelerated layout.** WebGPU compute shaders for 50,000+ node topologies.
2. **Streaming topology ingestion.** The timeline module (§15) supports file-based snapshot analysis; live gNMI/gRPC streaming would extend this to real-time updates.
3. **Simulated-annealing label post-pass.** Global optimization to resolve remaining overlaps in dense clusters.
4. **User study.** Controlled perceptual evaluation with network engineers.
5. **Collaborative editing.** The editor module (§18) provides single-user undo/redo; CRDT-based multi-user conflict resolution is the next step.
6. **SNMP and streaming telemetry adapters.** The Adapter trait is public; community adapters for SNMP, Napalm, and gNMI are encouraged.
7. **Parallel edge animation.** Traffic flow animation (§16) across parallel edge groups, showing per-link utilization within a LAG bundle.

References

- [1] NetBox Labs, *NetBox: The leading solution for network automation*, 2024. [Online]. Available: <https://netbox.dev>
- [2] E. R. Gansner and S. C. North, “An open graph visualization system and its applications to software engineering,” 11, vol. 30, 2000, pp. 1203–1233.

- [3] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel, “The open graph drawing framework (OGDF),” *Handbook of Graph Drawing and Visualization*, pp. 543–569, 2013.
- [4] M. Bostock, V. Ogievetsky, and J. Heer, “D3: Data-driven documents,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.
- [5] yWorks GmbH, *yEd graph editor*, 2024. [Online]. Available: <https://www.yworks.com/products/yed>
- [6] bluss, *Petgraph – graph data structure library for Rust*, 2024. [Online]. Available: <https://crates.io/crates/petgraph>
- [7] K. Sugiyama, S. Tagawa, and M. Toda, “Methods for visual understanding of hierarchical system structures,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 2, pp. 109–125, 1981.
- [8] T. M. J. Fruchterman and E. M. Reingold, “Graph drawing by force-directed placement,” *Software – Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991.
- [9] J. Barnes and P. Hut, “A hierarchical $O(N \log N)$ force-calculation algorithm,” *Nature*, vol. 324, pp. 446–449, 1986.
- [10] L. Verlet, “Computer “experiments” on classical fluids. I. thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, pp. 98–103, 1967.
- [11] J. Christensen, J. Marks, and S. Shieber, “An empirical study of algorithms for point-feature label placement,” in *ACM Transactions on Graphics*, vol. 14, 1995, pp. 203–232.
- [12] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, “The R*-tree: An efficient and robust access method for points and rectangles,” *ACM SIGMOD Record*, vol. 19, no. 2, pp. 322–331, 1990.
- [13] Stoeoef, *Rstar – R*-tree spatial index for Rust*, 2024. [Online]. Available: <https://crates.io/crates/rstar>
- [14] D. Holten and J. J. van Wijk, “Force-directed edge bundling for graph visualization,” *Computer Graphics Forum*, vol. 28, no. 3, pp. 983–990, 2009.
- [15] H. C. Purchase, “Metrics for graph drawing aesthetics,” *Journal of Visual Languages and Computing*, vol. 13, no. 5, pp. 501–516, 2002.
- [16] C. Dunne and B. Shneiderman, “Motif simplification: Improving network visualization readability with fan, connector, and clique glyphs,” *Proc. ACM CHI*, pp. 3247–3256, 2015.
- [17] J. Abello, F. van Ham, and N. Krishnan, “ASK-GraphView: A large scale graph visualization system,” in *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, 2006, pp. 669–676.
- [18] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

A Topology YAML Schema

Listing 33. Full YAML topology schema (abbreviated)

```

name: string          # Required: topology name
render:               # Optional: rendering hints
  preset: string       # Layout preset (default/sugiyama/radial/geo/subway)
  layout: string       # Layout algorithm (force_directed/hierarchical/
                        # radial/geographic/isometric/orthogonal/subway/tree)
  theme: string        # Theme (network/dark/light/blueprint/datacenter/print/high-contrast)
  width: number        # Canvas width in pixels
  height: number       # Canvas height in pixels
  lod: string          # LOD profile (none/conservative/balanced/aggressive/auto)
  seed: number         # RNG seed for deterministic layout
nodes:                # Required: list of nodes
  - id: string         # Unique node identifier
    label: string      # Display label (optional, defaults to id)
    node_type: string  # Device type (router/switch/server/firewall/cloud/
                        # l3_switch/load_balancer/database/endpoint/wireless_ap)
    attributes: map    # Arbitrary key-value metadata
    position:          # Position hint (optional)
      x: number
      y: number

```

```

group: string          # Group membership (optional)
layer: string          # Layer membership (optional, isometric)
highlight: string      # Effect name or true for default glow
interfaces:           # Interface list (optional)
  - id: string
    label: string
    status: string     # up/down/admin_down/errors
    mode: string       # dot/port/detailed
    marker: string     # none/dot/square/diamond
constraints:          # Layout constraints (optional)
  pin: { x: number, y: number }
  align_with: string  # node ID
  below: string       # node ID
  min_separation: number
edges:                # Required: list of edges
  - from: string      # Source node ID
    to: string        # Target node ID
    id: string        # Explicit edge ID (optional, for traffic/filter targeting)
    label: string     # Edge label (optional)
    weight: number    # Edge weight (default: 1.0)
    directed: boolean # Directed edge (default: false)
    bond_group: string # Bond/LAG group ID (optional)
    parallel_mode: string # offset/bundled/bonding_icon/stacked/angled
    gradient:         # Directional stroke gradient (optional)
      start_color: string # hex color at source
      end_color: string  # hex color at target
    attributes: map    # Arbitrary key-value metadata
groups:              # Optional: group definitions
  - id: string
    label: string
    style: map        # Boundary style overrides
layers:              # Optional: layer definitions for isometric layout
  - id: string
    label: string
    order: number     # Stacking order (lower = bottom)
paths:               # Optional: path emphasis definitions
  - id: string
    label: string
    emphasis: string  # primary/secondary
    fanout_mode: string # fixed/scaled/hybrid
    nodes: [string]   # Ordered list of node IDs
traffic:             # Optional: traffic flow animation overlay
  edges:
    - id: string      # Edge ID (matches edges[].id)
      utilization: number # 0.0--1.0 utilization level
      direction: string # forward/reverse/bidirectional
      style: string    # dot/dash animation style
annotations:         # Optional: SVG annotation markup
  - anchor: string    # Node ID to anchor near
    type: string      # text/callout
    text: string       # Annotation text
    shape: string      # circle/box/arrow (callout only)
    layer: string      # Annotation layer name

```

B Configuration Reference

B.1 CLI Flags

B.1.1 Input and Output

Flag	Type	Default	Description
--input, -i	path	(required)	Topology file; globs and - for stdin
--output, -o	path	auto	Output file path
--config	path	—	JSON config file
--format	string	svg	Output format: svg, png, pdf, html, dot
--dpi	integer	96	DPI for PNG export

B.1.2 Layout

Flag	Default	Description
--layout	auto	Algorithm: force-directed, hierarchical, radial, isometric, geographic, orthogonal, subway, tree
--preset	—	Layout preset: default, wan, datacenter, sugiyama, geographic, subway, ...
--template	—	Topology template: spine-leaf, hub-spoke, ring, multi-tier
--seed	42	RNG seed for deterministic layout

B.1.3 Appearance

Flag	Default	Description
--theme	network	Theme (7 built-in): light, dark, network, blueprint, datacenter, print, high-contrast
--labels	label	Label mode: off, label, id
--node-size	auto	Node radius in scene units
--lod	auto	LOD profile: none, conservative, balanced, aggressive, auto

B.1.4 Canvas and Viewport

Flag	Default	Description
--width	1200	SVG viewport width (px)
--height	800	SVG viewport height (px)
--padding	40	ViewBox padding (px)
--padding-ratio	0.06	Padding as fraction of scene

B.1.5 Force-Directed Tuning

Flag	Default	Description
--link-distance	30.0	Target link length (scene units)
--link-strength	auto	Edge spring strength (0.0–1.0)
--charge-strength	−30.0	Many-body charge (negative = repel)
--max-iterations	300	Maximum simulation iterations
--soft-anchor-strength	0.0	Anchor strength for hinted nodes
--refine	false	Apply post-layout edge refinement

B.1.6 Geographic Layout

Flag	Default	Description
--show-map	false	Show background coastline map
--great-circle-arcs	false	Render edges as curved arcs
--show-distances	false	Show geodesic distance labels
--distance-unit	km	Distance unit: km or mi

B.1.7 Filtering

Flag	Description
--show-type	Only render nodes of these types (comma-separated, repeatable)
--hide-type	Hide nodes of these types
--show-tag	Only render nodes with these tags
--hide-tag	Hide nodes with any of these tags
--show-group	Only render nodes in these groups
--hide-group	Hide nodes in these groups
--filter-file	Load filter rules from YAML file
--path	BFS path query between two node IDs
--highlight-path	Highlight a named path (repeatable)

B.1.8 Analysis Features

Flag	Description
--diff <path>	Compare against a base topology and visualize changes
--traffic <path>	Traffic overlay configuration YAML
--annotations <path>	Annotation markup YAML
--hide-layers	Suppress named annotation layers
--show-layers	Force-show named annotation layers
--cluster	Enable automatic community detection
--algorithm	Clustering algorithm: louvain, label-propagation, edge-betweenness
--metric <field>	Map a node attribute to color (see §10)
--palette <name>	Color palette: blues, greens, oranges, reds, purples, greys
--analytics	Compute and display graph analytics

B.1.9 Quality and Diagnostics

Flag	Default	Description
--quality-threshold	85.0	Quality score warning threshold
--fail-on-low-quality	false	Exit with error if below threshold
--deterministic	true	Deterministic output mode
--diagnostics	—	Structured diagnostics: json, yaml, ndjson
--verbose, -v	false	Show debug statistics

B.2 CLI Subcommands

Command	Description
list-examples	List built-in topology examples
print-example <name>	Print a built-in topology to stdout
list-templates	List layout templates
list-adapters	List data source adapters
import	Import topology from adapter into YAML
schema	Print JSON Schema (config, topology, or diagnostics)
print-config <name>	Print an example config to stdout
validate	Validate topology/config against schema
migrate	Normalize files to current schema version
check-quality	Analyze SVG files for quality issues
diff	Compare two topologies, visualize changes
show	Render topology to ASCII art in the terminal

B.3 JSON Configuration File

The `--config` flag accepts a JSON file with the following structure:

Listing 34. NetVisConfig JSON

```
{
  "schema_version": "1.0",
  "layout": "force_directed",
  "quality": "balanced",
  "width": 1200.0,
  "height": 800.0,
  "effects": {
    "max_filtered_elements": 50
  }
}
```

Key	Type	Default	Description
layout	string	force_directed	Layout algorithm
quality	string	balanced	Quality preset: quick, balanced, quality
width	float	800.0	Viewport width
height	float	600.0	Viewport height
effects.max_filtered_elements	int	50	SVG filter budget

Table 7. JSON configuration keys.

Quality presets control iteration multipliers: quick (0.33x), balanced (1.0x), quality (3.0x).

Index

adapters, 12
AestheticScore, 30
annotation, 27
API, 33
architecture, 12

Barnes-Hut, 14
benchmarks, 35
betweenness centrality, 29
bond group, 17

callout, 27
clarity score, 30
CLI, 33
CLI flags, 39
clustering coefficient, 29
collision avoidance, 23
color palettes, 22
configuration, 39
crossings, 30
CSS animation, 26

data model, 11
data-driven visualization, 22
deployment, 35
device icons, 18
diff, 26
Docker, 35
drop shadow, 27

edge, 11
edge bundling, 24
edge gradients, 21
editor, 27
EffectRegistry, 27, 33

FDEB, 24
filter DSL, 29
flow direction, 21
force-directed edge bundling, 24
fuzzy matching, 30

glow, 27
graph analytics, 29

HTML export, 31

interface status, 20
interfaces, 20

JSON config, 39
label placement, 23
LAG, 17

layout
 composite, 14
 force-directed, 14
 geographic, 14
 hierarchical, 14
 isometric, 14
 orthogonal, 14, 15
 radial, 14
 subway, 14, 16
 tree, 14, 16
link aggregation, 17

metric encoding, 22

node, 11
node renderer, 33
node shapes, 18
node_type, 18
nudging, 25

octilinear, 16
overlap, 30

parallel edges, 17
path emphasis, 21
PathFanoutMode, 21
PDF, 31
performance, 35
pipeline, 12
plugin, 33
PNG, 31
ports, 20

quality metrics, 30

R*-tree, 23
refiners, 25
renderer, 12
rendering pipeline, 31
routing
 orthogonal, 25
 port-aware, 25
 smart, 25

scene graph, 11
snapshot, 26
Sugiyama, 14
SVG, 31
SVG markup, 27

temporal analysis, 26
timeline, 26
topology diffing, 26
topology filtering, 29

topology graph, [11](#)
traffic animation, [26](#)
typo correction, [30](#)

undo/redo, [27](#)
utilization, [26](#)

validation, [30](#)
visual effects, [27](#)

WASM, [27](#)
WASM API, [33](#)
WebAssembly, [35](#)

DRAFT